

+ 26

Senders, Receivers, and Robots

Structured Concurrency for Autonomous Navigation

NAHUEL ESPINOSA





NAHUEL ESPINOSA

pronounced: /*nah-well*/

Robotics Software Engineer



MOTIVATION

We want robots to do things,

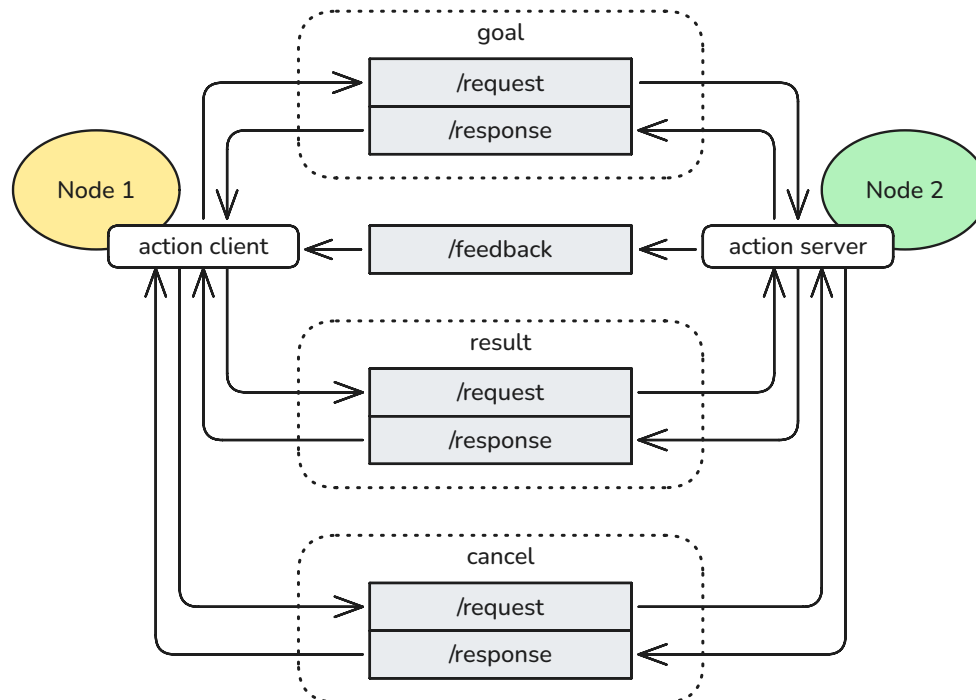
MOTIVATION

We want robots to do things,
and we also want the ability to stop them.

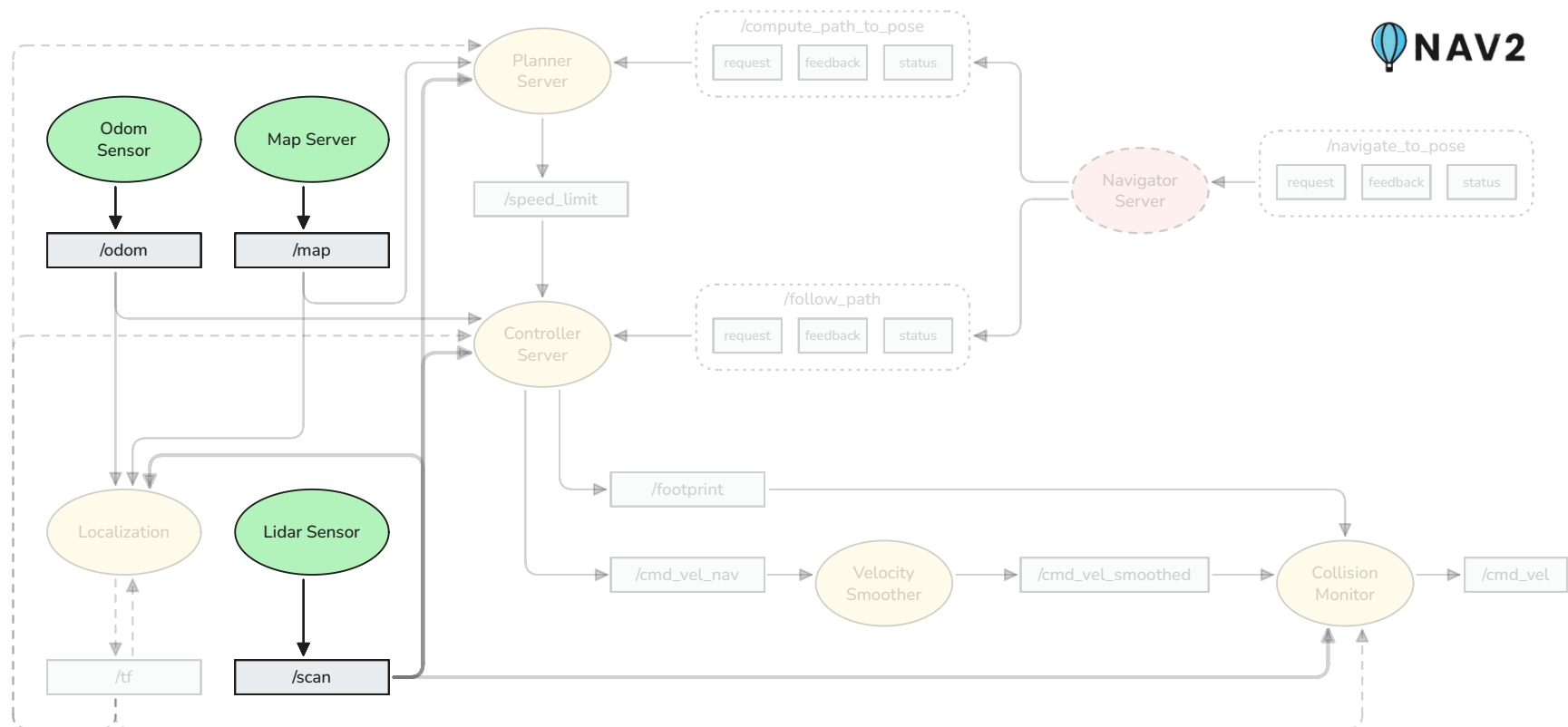
MOTIVATION

Actions are preemptible, long-running tasks.

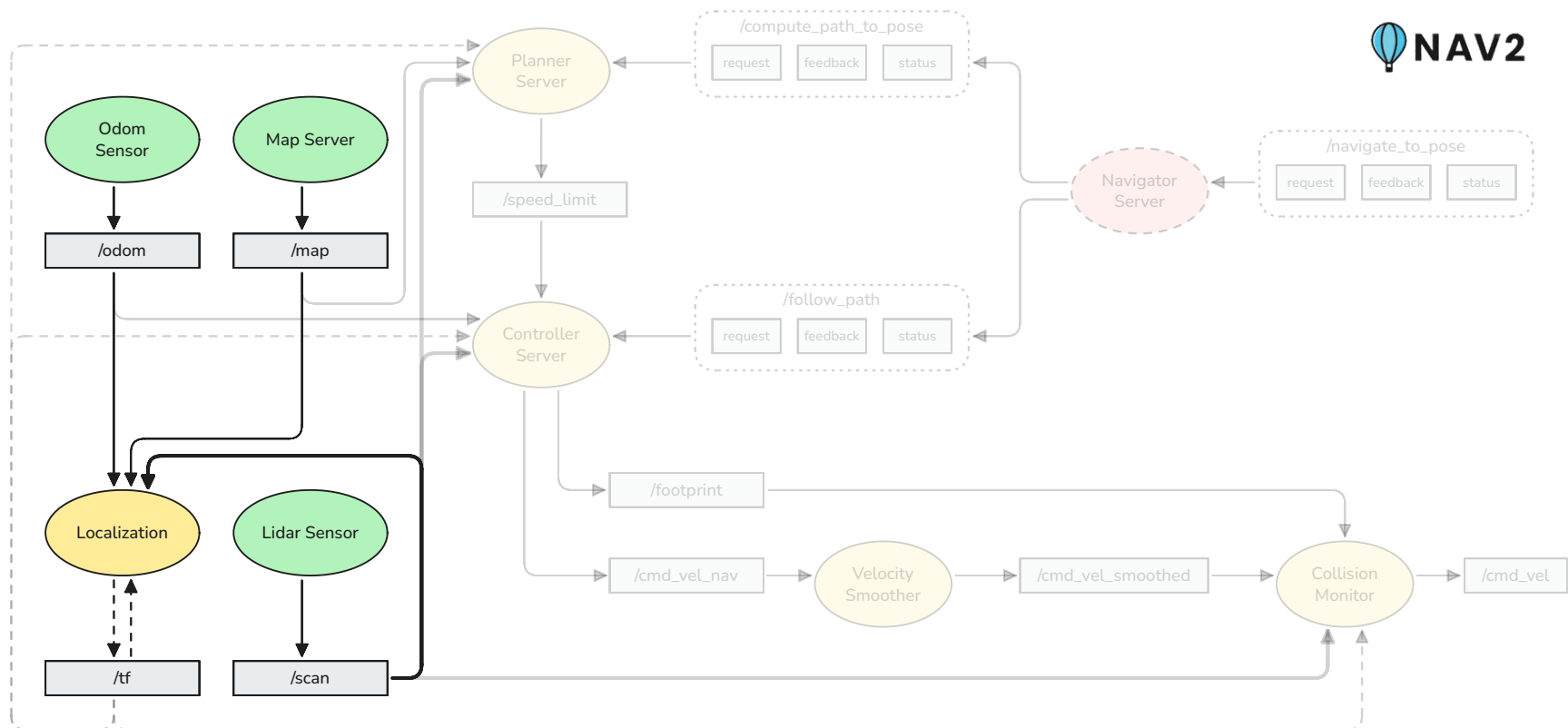
ROS2



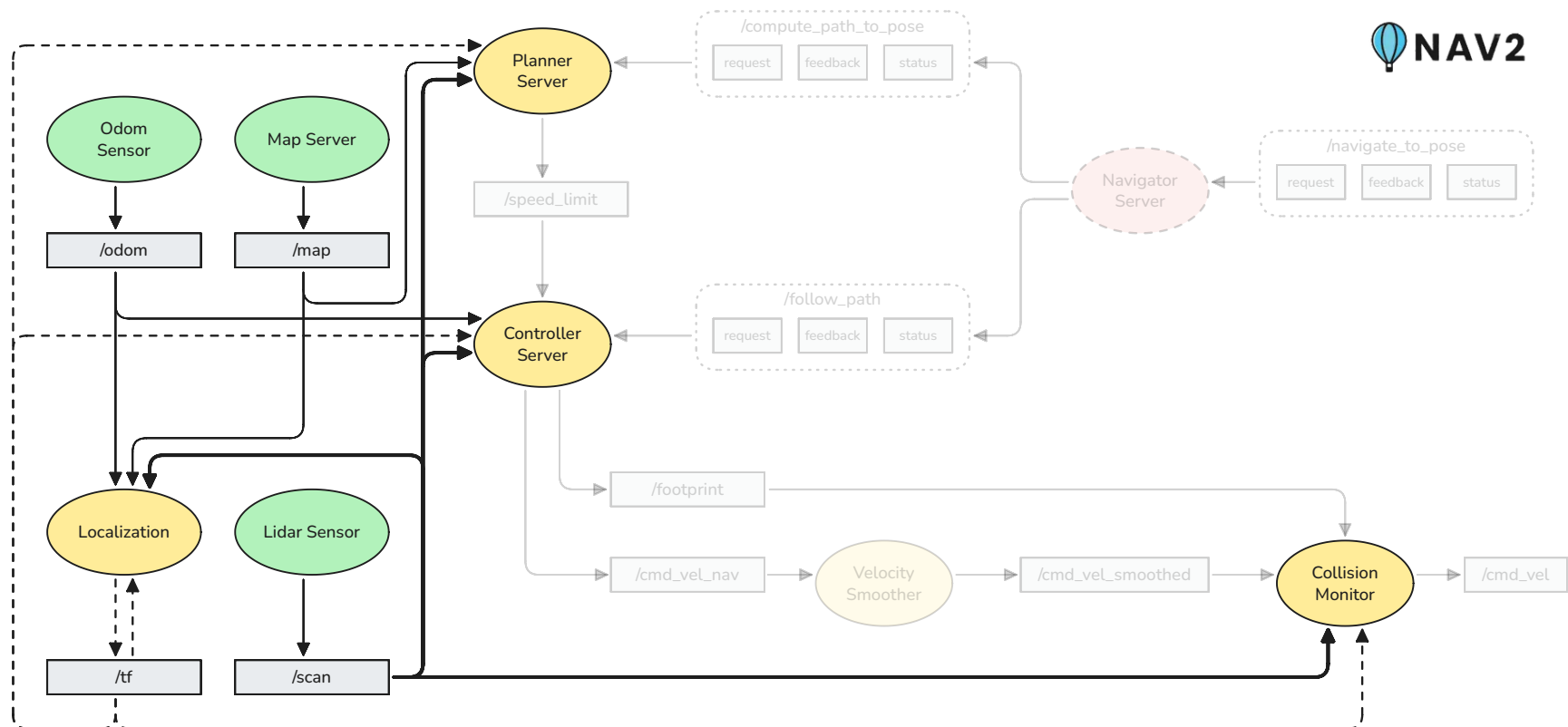
MOTIVATION



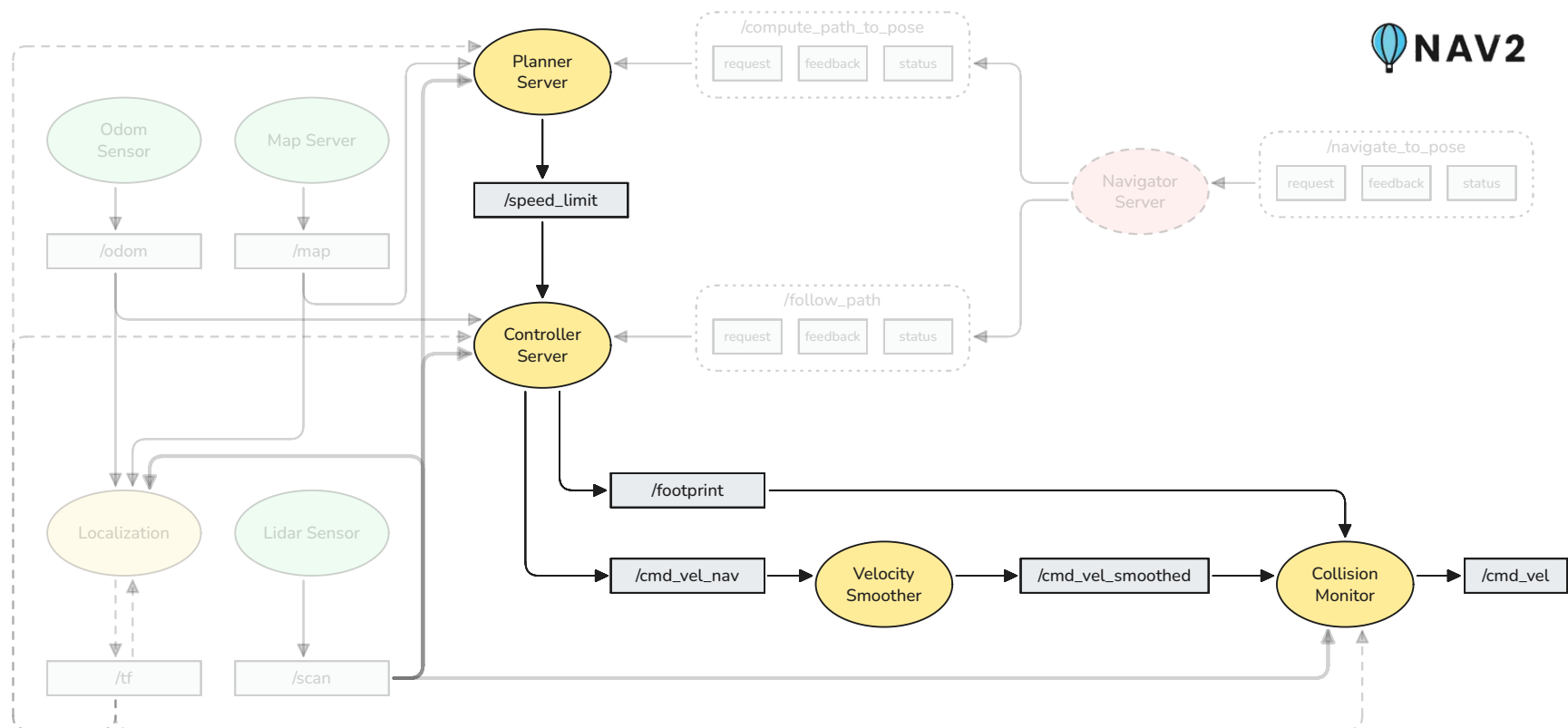
MOTIVATION



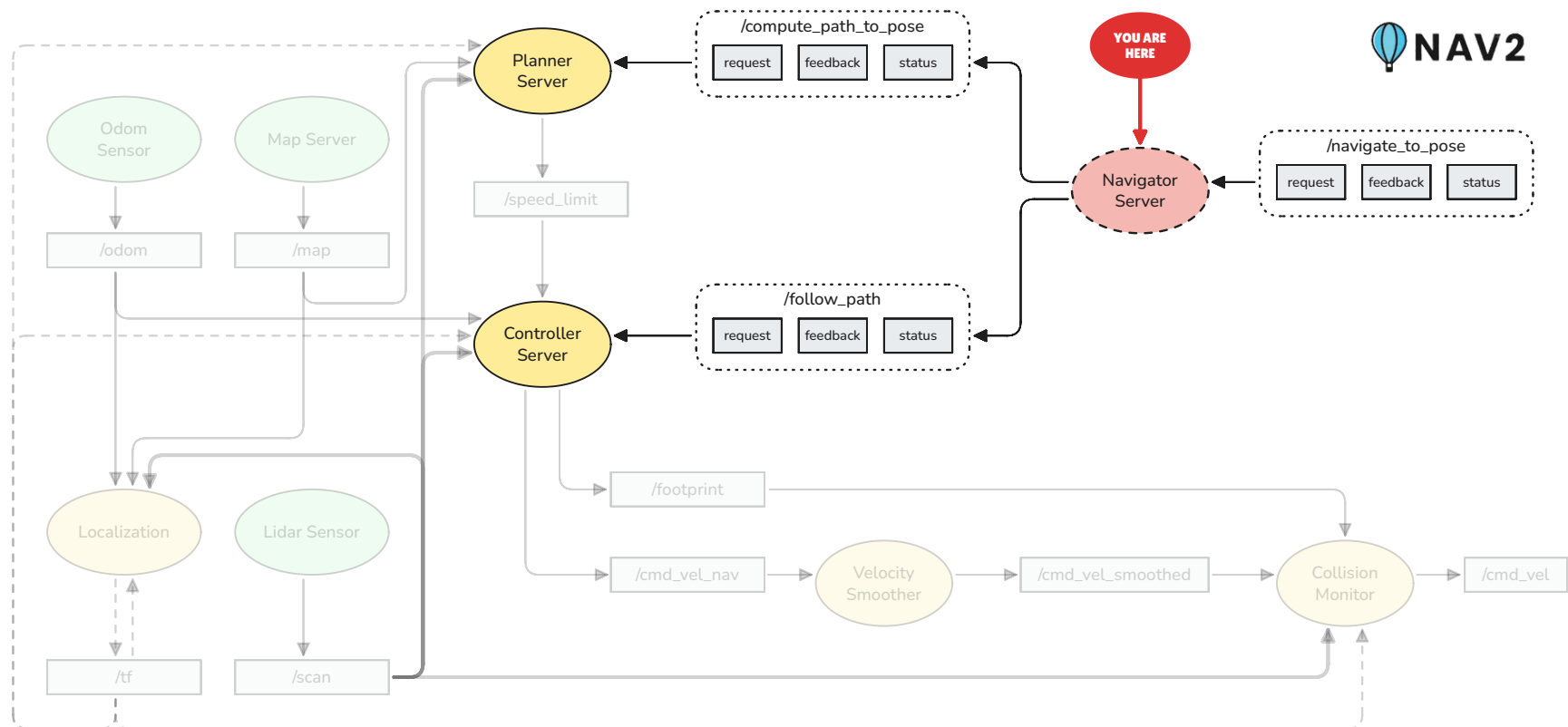
MOTIVATION



MOTIVATION



MOTIVATION



MOTIVATION

```
1 template <typename T>
2 class action_client {
3     void async_send_goal(
4         const goal&,
5         std::function<void(std::shared_ptr<goal_handle>)>>);
6
7     void async_get_result(
8         std::shared_ptr<goal_handle>,
9         std::function<void(const wrapped_result&)>);
10
11     void async_cancel_goal(
12         std::shared_ptr<goal_handle>);
13 };
```

MOTIVATION

```
1 template <typename T>
2 class action_client {
3     void async_send_goal(
4         const goal&,
5         std::function<void(std::shared_ptr<goal_handle>)>>);
6
7     void async_get_result(
8         std::shared_ptr<goal_handle>,
9         std::function<void(const wrapped_result&)>);
10
11     void async_cancel_goal(
12         std::shared_ptr<goal_handle>);
13 };
```

MOTIVATION

```
1 template <typename T>
2 class action_client {
3     void async_send_goal(
4         const goal&,
5         std::function<void(std::shared_ptr<goal_handle>)>>);
6
7     void async_get_result(
8         std::shared_ptr<goal_handle>,
9         std::function<void(const wrapped_result&)>);
10
11     void async_cancel_goal(
12         std::shared_ptr<goal_handle>);
13 };
```

MOTIVATION

```
1 template <typename T>
2 class action_client {
3     void async_send_goal(
4         const goal&,
5         std::function<void(std::shared_ptr<goal_handle>)>>);
6
7     void async_get_result(
8         std::shared_ptr<goal_handle>,
9         std::function<void(const wrapped_result&)>);
10
11     void async_cancel_goal(
12         std::shared_ptr<goal_handle>);
13 };
```

MOTIVATION

CHALLENGE

```
1 class navigator {  
2     public:  
3         void async_navigate(const auto& planner_goal);  
4  
5     private:  
6         std::shared_ptr<action_client<plan_action>> planner_;  
7         std::shared_ptr<action_client<control_action>> controller_;  
8         std::shared_ptr<timer> timer_;  
9 };
```

1. Call the planning action.
2. Pass the result to the control action.
3. Enforce a 60-second timeout on the controller.

MOTIVATION

1. Call the planning action.

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_->async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             if (!handle) {
6                 report_failure("planner rejected goal");
7                 return;
8             }
9
10            planner_->async_get_result(
11                handle,
12                [this](const auto& result) {
13                    ...
14                });
15        });
16 }
```



MOTIVATION

1. Call the planning action.

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_->async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             if (!handle) {
6                 report_failure("planner rejected goal");
7                 return;
8             }
9
10            planner_->async_get_result(
11                handle,
12                [this](const auto& result) {
13                    ...
14                });
15        });
16 }
```



MOTIVATION

1. Call the planning action.

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_->async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             if (!handle) {
6                 report_failure("planner rejected goal");
7                 return;
8             }
9
10            planner_->async_get_result(
11                handle,
12                [this](const auto& result) {
13                    ...
14                });
15        });
16 }
```



MOTIVATION

1. Call the planning action.

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_->async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             if (!handle) {
6                 report_failure("planner rejected goal");
7                 return;
8             }
9
10            planner_->async_get_result(
11                handle,
12                [this](const auto& result) {
13                    ...
14                });
15        });
16 }
```



MOTIVATION

2. Pass the result to the control action.

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_>async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             if (!handle) {
6                 report_failure("planner rejected goal");
7                 return;
8             }
9
10            planner_>async_get_result(
11                handle,
12                [this](const auto& result) {
13                    controller_>async_send_goal(
14                        get_controller_goal_from_planner_result(result),
15                        [this](auto handle) {
16                            if (!handle) {
17                                report_failure("controller rejected goal");
18                                return;
19                            }
20
21                            controller_>async_get_result(
22                                handle,
23                                [this](const auto& result) {
24                                    report_success();
25                                });
26                        });
27                });
28    });
29 }
```



MOTIVATION

2. Pass the result to the control action.

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_>async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             if (!handle) {
6                 report_failure("planner rejected goal");
7                 return;
8             }
9
10            planner_>async_get_result(
11                handle,
12                [this](const auto& result) {
13                    controller_>async_send_goal(
14                        get_controller_goal_from_planner_result(result),
15                        [this](auto handle) {
16                            if (!handle) {
17                                report_failure("controller rejected goal");
18                                return;
19                            }
20
21                            controller_>async_get_result(
22                                handle,
23                                [this](const auto& result) {
24                                    report_success();
25                                });
26                        });
27                });
28        });
29 }
```



MOTIVATION

2. Pass the result to the control action.

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_>async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             if (!handle) {
6                 report_failure("planner rejected goal");
7                 return;
8             }
9
10            planner_>async_get_result(
11                handle,
12                [this](const auto& result) {
13                    controller_>async_send_goal(
14                        get_controller_goal_from_planner_result(result),
15                        [this](auto handle) {
16                            if (!handle) {
17                                report_failure("controller rejected goal");
18                                return;
19                            }
20
21                            controller_>async_get_result(
22                                handle,
23                                [this](const auto& result) {
24                                    report_success();
25                                });
26                        });
27                });
28        });
29 }
```



MOTIVATION

3. Enforce a 60-second timeout on the controller.

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_>async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             if (!handle) {
6                 report_failure("planner rejected goal");
7                 return;
8             }
9
10            planner_>async_get_result(
11                handle,
12                [this](const auto& result) {
13                    controller_>async_send_goal(
14                        get_controller_goal_from_planner_result(result),
15                        [this](auto handle) {
16                            if (!handle) {
17                                report_failure("controller rejected goal");
18                                return;
19                            }
20
21                            timer_.start(60s, [this, handle]() {
22                                controller_>async_cancel_goal(handle);
23                            });
24
25                            controller_>async_get_result(
26                                handle,
27                                [this](const auto& result) {
28                                    timer_.stop();
29                                    if (result.code == result_code::CANCELED) {
30                                        report_failure("controller timed out");
31                                        return;
32                                    }

```

```
33         report_success();
34     });
35 });
36
```

MOTIVATION

3. Enforce a 60-second timeout on the controller.

```
3 planner_goal,  
4 [this](auto handle) {  
5     if (!handle) {  
6         report_failure("planner rejected goal");  
7         return;  
8     }  
9  
10    planner_>async_get_result(  
11        handle,  
12        [this](const auto& result) {  
13            controller_>async_send_goal(  
14                get_controller_goal_from_planner_result(result),  
15                [this](auto handle) {  
16                    if (!handle) {  
17                        report_failure("controller rejected goal");  
18                        return;  
19                    }  
20  
21                    timer_.start(60s, [this, handle]() {  
22                        controller_>async_cancel_goal(handle);  
23                    });  
24  
25                    controller_>async_get_result(  
26                        handle,  
27                        [this](const auto& result) {  
28                            timer_.stop();  
29                            if (result.code == result_code::CANCELED) {  
30                                report_failure("controller timed out");  
31                                return;  
32                            }  
33                        })  
34                    }  
35                })  
36            }  
37        })  
38    );  
39}
```

```
36         });  
37     });  
38     // ...  
36     // ...
```

MOTIVATION

3. Enforce a 60-second timeout on the controller.

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_>async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             if (!handle) {
6                 report_failure("planner rejected goal");
7                 return;
8             }
9
10            planner_>async_get_result(
11                handle,
12                [this](const auto& result) {
13                    controller_>async_send_goal(
14                        get_controller_goal_from_planner_result(result),
15                        [this](auto handle) {
16                            if (!handle) {
17                                report_failure("controller rejected goal");
18                                return;
19                            }
20
21                            timer_.start(60s, [this, handle]() {
22                                controller_>async_cancel_goal(handle);
23                            });
24
25                            controller_>async_get_result(
26                                handle,
27                                [this](const auto& result) {
28                                    timer_.stop();
29                                    if (result.code == result_code::CANCELED) {
30                                        report_failure("controller timed out");
31                                        return;
32                                    }

```

```
33         report_success();
34     });
35 });
36 ..
```

MOTIVATION

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_>async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             on_planner_goal_response(handle);
6         });
7 }
8
9 void navigator::on_planner_goal_response(auto handle) {
10     if (!handle) {
11         report_failure("planner rejected goal");
12         return;
13     }
14
15     planner_>async_get_result(
16         handle,
17         [this](const auto& result) {
18             on_planner_result(result);
19         });
20 }
21
22 void navigator::on_planner_result(const auto& result) {
23     controller_>async_send_goal(
24         get_controller_goal_from_planner_result(result),
25         [this](auto handle) {
26             on_controller_goal_response(handle);
27         });
28 }
29
30 void navigator::on_controller_goal_response(auto handle) { ... }
31
32 void navigator::on_controller_result(const auto& result) { ... }
```


MOTIVATION

```
1 void navigator::async_navigate(const auto& planner_goal) {
2     planner_>async_send_goal(
3         planner_goal,
4         [this](auto handle) {
5             on_planner_goal_response(handle);
6         });
7 }
8
9 void navigator::on_planner_goal_response(auto handle) {
10     if (!handle) {
11         report_failure("planner rejected goal");
12         return;
13     }
14
15     planner_>async_get_result(
16         handle,
17         [this](const auto& result) {
18             on_planner_result(result);
19         });
20 }
21
22 void navigator::on_planner_result(const auto& result) {
23     controller_>async_send_goal(
24         get_controller_goal_from_planner_result(result),
25         [this](auto handle) {
26             on_controller_goal_response(handle);
27         });
28 }
29
30 void navigator::on_controller_goal_response(auto handle) { ... }
31
32 void navigator::on_controller_result(const auto& result) { ... }
```


MOTIVATION

```
1 void navigator::async_navigate(const auto& planner
2   planner_->async_send_goal(
3     planner_goal,
4     [this](auto handle) {
5       on_planner_goal_response(handle);
6     });
7 }
8
9 void navigator::on_planner_goal_response(auto handle) {
10   if (!handle) {
11     report_failure("planner rejected goal");
12     return;
13   }
14
15   planner_->async_get_result(
16     handle,
17     [this](const auto& result) {
18       on_planner_result(result);
19     });
20 }
21
22 void navigator::on_planner_result(const auto& result) {
23   controller_->async_send_goal(
24     get_controller_goal_from_planner_result(result),
25     [this](auto handle) {
26       on_controller_goal_response(handle);
27     });
28 }
29
30 void navigator::on_controller_goal_response(auto handle) {
31
32 void navigator::on_controller_result(const auto& result) {
```

1. Fragmented Logic:

The control flow is scattered across multiple methods.



MOTIVATION

```
1 void navigator::async_navigate(const auto& planner
2   planner_->async_send_goal(
3     planner_goal,
4     [this](auto handle) {
5       on_planner_goal_response(handle);
6     });
7 }
8
9 void navigator::on_planner_goal_response(auto handle) {
10   if (!handle) {
11     report_failure("planner rejected goal");
12     return;
13   }
14
15   planner_->async_get_result(
16     handle,
17     [this](const auto& result) {
18       on_planner_result(result);
19     });
20 }
21
22 void navigator::on_planner_result(const auto& result) {
23   controller_->async_send_goal(
24     get_controller_goal_from_planner_result(result),
25     [this](auto handle) {
26       on_controller_goal_response(handle);
27     });
28 }
29
30 void navigator::on_controller_goal_response(auto handle) {
31
32 void navigator::on_controller_result(const auto& result) {
```

1. Fragmented Logic:

The control flow is scattered across multiple methods.

2. State Management:

Local variables don't survive, so we need to persist state in other ways.



MOTIVATION

```
1 void navigator::async_navigate(const auto& planner
2   planner_->async_send_goal(
3     planner_goal,
4     [this](auto handle) {
5       on_planner_goal_response(handle);
6     });
7 }
8
9 void navigator::on_planner_goal_response(auto handle) {
10   if (!handle) {
11     report_failure("planner rejected goal");
12     return;
13   }
14
15   planner_->async_get_result(
16     handle,
17     [this](const auto& result) {
18       on_planner_result(result);
19     });
20 }
21
22 void navigator::on_planner_result(const auto& result) {
23   controller_->async_send_goal(
24     get_controller_goal_from_planner_result(result),
25     [this](auto handle) {
26       on_controller_goal_response(handle);
27     });
28 }
29
30 void navigator::on_controller_goal_response(auto handle) {
31
32 void navigator::on_controller_result(const auto& result) {
```

1. Fragmented Logic:

The control flow is scattered across multiple methods.

2. State Management:

Local variables don't survive, so we need to persist state in other ways.

3. Error Handling:

We are forced to manually trigger failure routines.



MOTIVATION

```
1 void navigator::async_navigate(const auto& planner
2   planner_->async_send_goal(
3     planner_goal,
4     [this](auto handle) {
5       on_planner_goal_response(handle);
6     });
7 }
8
9 void navigator::on_planner_goal_response(auto handle) {
10   if (!handle) {
11     report_failure("planner rejected goal");
12     return;
13   }
14
15   planner_->async_get_result(
16     handle,
17     [this](const auto& result) {
18       on_planner_result(result);
19     });
20 }
21
22 void navigator::on_planner_result(const auto& result) {
23   controller_->async_send_goal(
24     get_controller_goal_from_planner_result(result),
25     [this](auto handle) {
26       on_controller_goal_response(handle);
27     });
28 }
29
30 void navigator::on_controller_goal_response(auto handle) {
31
32 void navigator::on_controller_result(const auto& result) {
```

1. Fragmented Logic:

The control flow is scattered across multiple methods.

2. State Management:

Local variables don't survive, so we need to persist state in other ways.

3. Error Handling:

We are forced to manually trigger failure routines.

4. Cancellation:

Requires manual handling.



MOTIVATION

ARE CALLBACKS BAD?

MOTIVATION

ARE CALLBACKS BAD?

No.

They are the *oldest, simplest, most powerful,*
and most efficient mechanism for creating chains of work.

MOTIVATION

ARE CALLBACKS BAD?

No.

They are the *oldest, simplest, most powerful, and most efficient* mechanism for creating chains of work.

But when used to structure **application logic**, we are forced to manually solve fundamental asynchronous problems in the middle of our code:

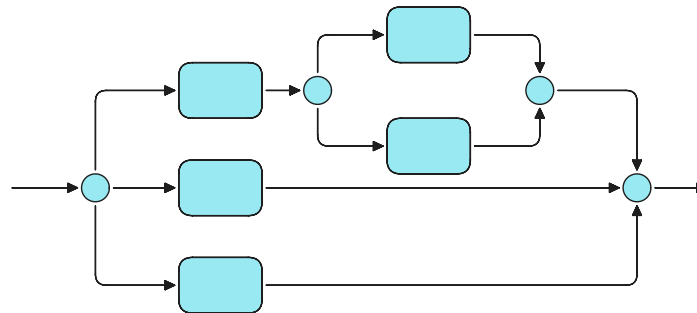
- State propagation
- Error bubbling
- Cancellation
- Synchronization

AGENDA

1. Structured concurrency
2. `std::future?`
3. `std::exec::sender`
4. Adapter layer
5. Case study: Navigator
6. Live demo!

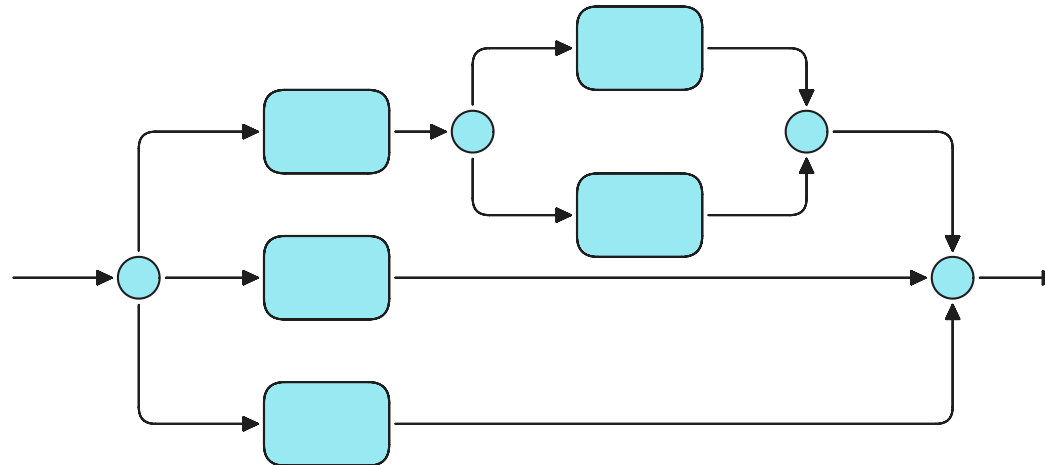
STRUCTURED CONCURRENCY

- (2016) [Structured Concurrency](#) by Martin Sústrik
- (2017) [Notes on structured concurrency, or: Go statement considered harmful](#) by Nathaniel J. Smith
- (2020) [Structured Concurrency](#) by Eric Niebler



[...] a way to structure async computations so that child operations are guaranteed to complete before their parents, just like a normal function.

STRUCTURED CONCURRENCY



Lifetimes are tied to the scope

Control Flow has clear entry and exit points

Errors propagate from children to parents

Cancellation is requested from parents to children

STRUCTURED CONCURRENCY

This concept can be applied to *any* async API.

Async Framework	Async Application
-----------------	-------------------

- | | |
|--|---|
| <ul style="list-style-type: none">• Control flow | <ul style="list-style-type: none">• Lifetimes• Error handling• Cancellation |
|--|---|

STRUCTURED CONCURRENCY

This concept can be applied to *any* async API.

Async Framework	Async Application
-----------------	-------------------

- | | |
|---|--|
| <ul style="list-style-type: none">• Lifetimes• Error handling• Cancellation | <ul style="list-style-type: none">• Control flow |
|---|--|

FUTURES?

Introduced in C++11 to represent
the result of async operations.

FUTURES?

Introduced in C++11 to represent
the result of async operations.

```
1 template <typename T>
2 class action_client {
3     auto async_send_goal(const goal&) -> std::shared_future<std::shared_ptr<goal_handle>>;
4
5     auto async_get_result(std::shared_ptr<goal_handle>) -> std::shared_future<wrapped_result>;
6
7     auto async_cancel_goal(std::shared_ptr<goal_handle>) -> std::shared_future<void>;
8 };
```

FUTURES?

```
1 auto handle = controller->async_send_goal(plan_result).get();
2 if (controller->async_get_result(handle).wait_for(60s) == std::future_status::timeout) {
3     controller->async_cancel_goal(handle).get();
4 }
```

FUTURES?

```
1 auto handle = controller->async_send_goal(plan_result).get();  
2 if (controller->async_get_result(handle).wait_for(60s) == std::future_status::timeout) {  
3     controller->async_cancel_goal(handle).get();  
4 }
```

FUTURES?

```
1 auto handle = controller->async_send_goal(plan_result).get();  
2 if (controller->async_get_result(handle).wait_for(60s) == std::future_status::timeout) {  
3     controller->async_cancel_goal(handle).get();  
4 }
```

FUTURES?

```
1 auto handle = controller->async_send_goal(plan_result).get();  
2 if (controller->async_get_result(handle).wait_for(60s) == std::future_status::timeout) {  
3     controller->async_cancel_goal(handle).get();  
4 }
```

FUTURES?

```
1 auto handle = controller->async_send_goal(plan_result).get();  
2 if (controller->async_get_result(handle).wait_for(60s) == std::future_status::timeout) {  
3     controller->async_cancel_goal(handle).get();  
4 }
```

- Block the current thread.

FUTURES?

```
1 auto handle = controller->async_send_goal(plan_result).get();  
2 if (controller->async_get_result(handle).wait_for(60s) == std::future_status::timeout) {  
3     controller->async_cancel_goal(handle).get();  
4 }
```

- Block the current thread.
- Do not compose well.

FUTURES?

```
1 auto handle = controller->async_send_goal(plan_result).get();  
2 if (controller->async_get_result(handle).wait_for(60s) == std::future_status::timeout) {  
3     controller->async_cancel_goal(handle).get();  
4 }
```

- Block the current thread.
- Do not compose well.
- Have hidden overhead.

SENDERS

- Standardized as part of the execution control library (since C++26) - [P2300](#)

SENDERS

- Standardized as part of the execution control library (since C++26) - [P2300](#)
- Support error handling seamlessly.

SENDERS

- Standardized as part of the execution control library (since C++26) - [P2300](#)
- Support error handling seamlessly.
- (Optionally) Support cancellation.

SENDERS

- Standardized as part of the execution control library (since C++26) - [P2300](#)
- Support error handling seamlessly.
- (Optionally) Support cancellation.
- Highly composable.

SENDERS

- Standardized as part of the execution control library (since C++26) - [P2300](#)
- Support error handling seamlessly.
- (Optionally) Support cancellation.
- Highly composable.
- Highly customizable.

SENDERS

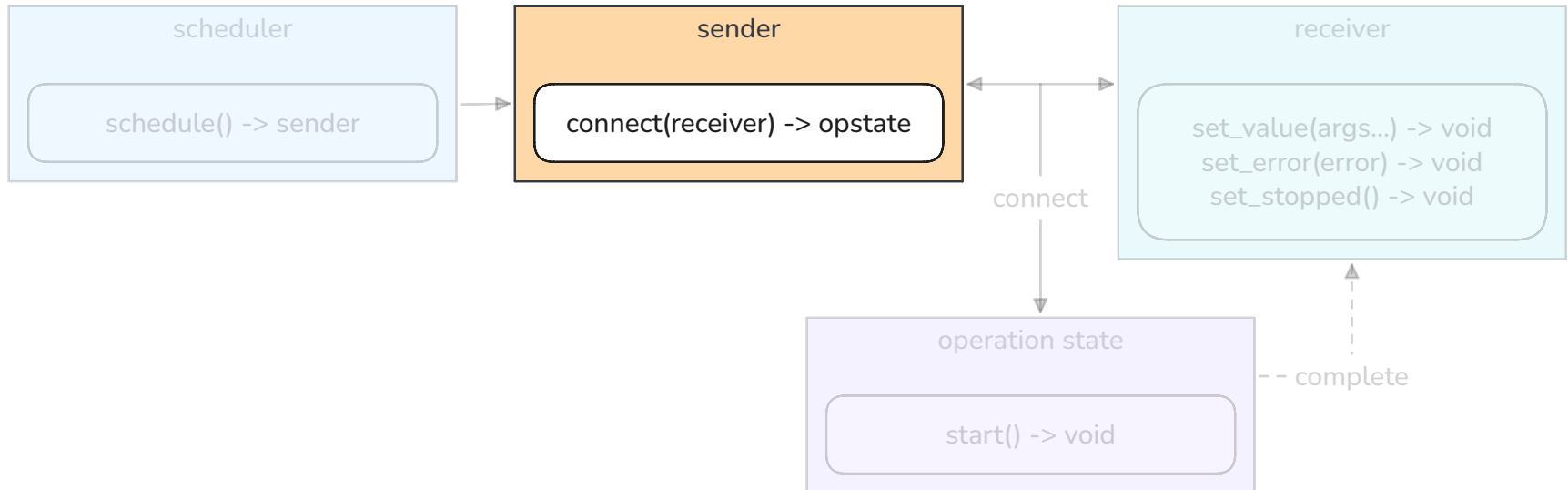
- Standardized as part of the execution control library (since C++26) - [P2300](#)
- Support error handling seamlessly.
- (Optionally) Support cancellation.
- Highly composable.
- Highly customizable.
- Enable the implementation of generic asynchronous algorithms.

SENDERS

- Standardized as part of the execution control library (since C++26) - [P2300](#)
- Support error handling seamlessly.
- (Optionally) Support cancellation.
- Highly composable.
- Highly customizable.
- Enable the implementation of generic asynchronous algorithms.

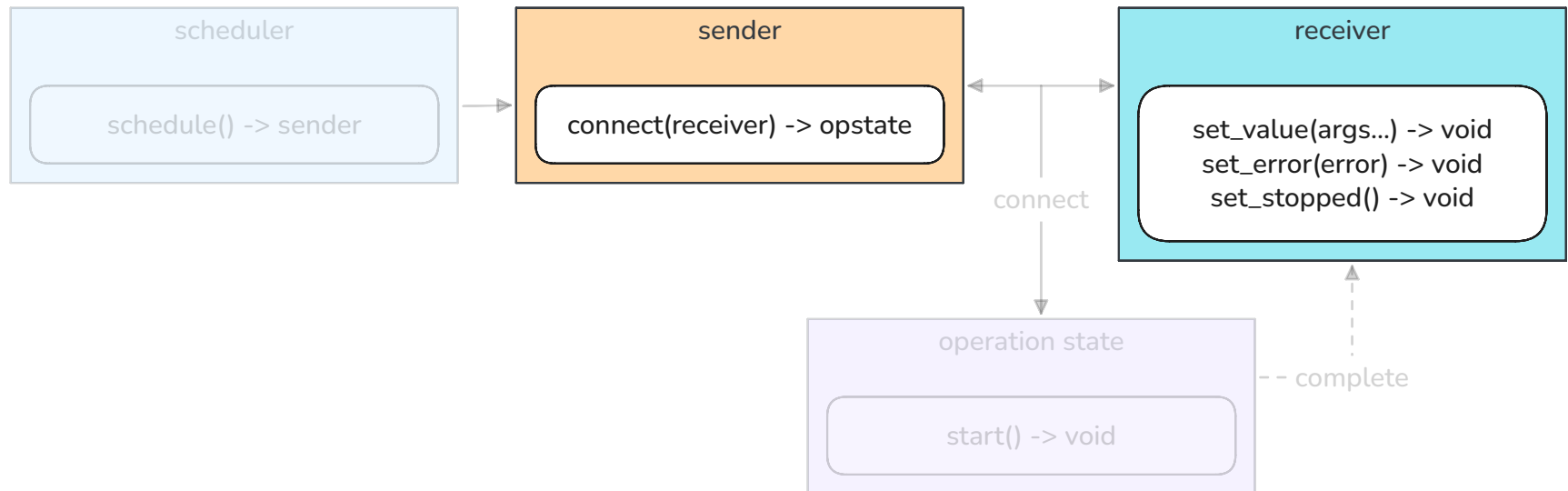
"Senders end callback hell"

SENDERS



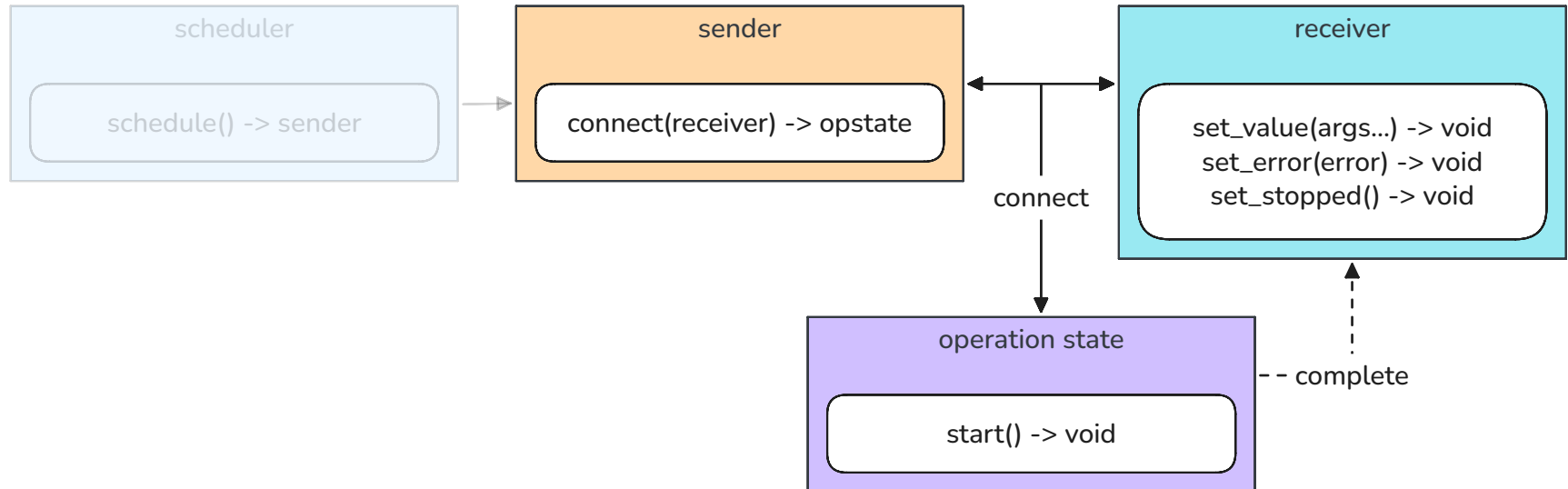
- A *sender* is an object that describes a unit of async work.

SENDERS



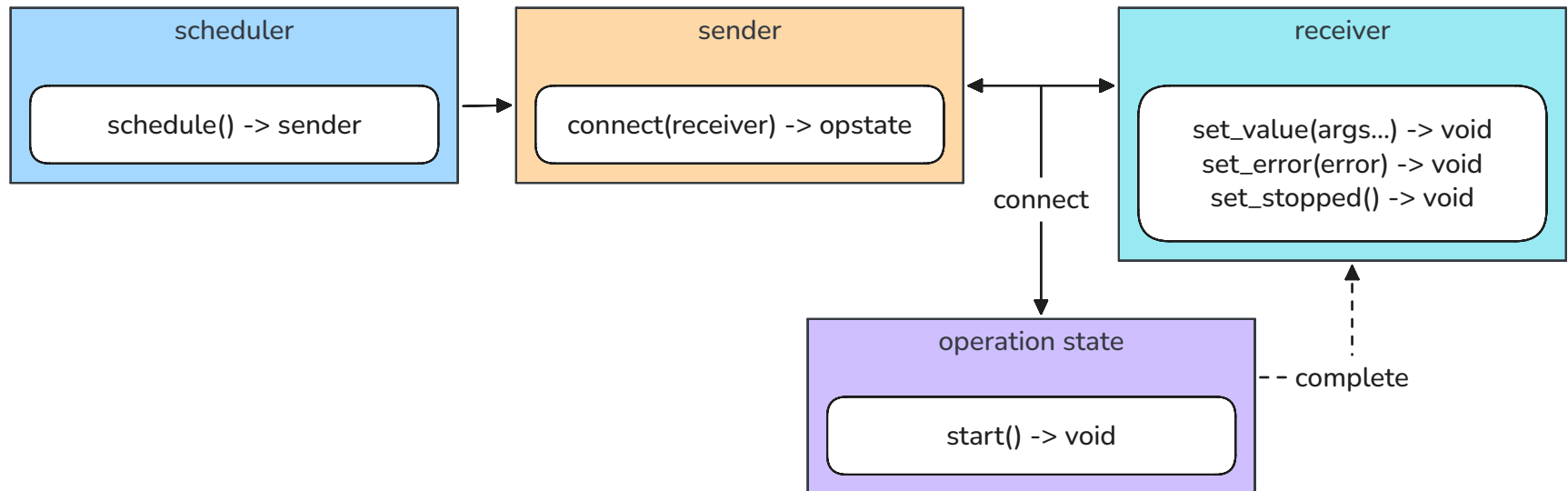
- A *sender* is an object that describes a unit of async work.
- A *receiver* is an object that handles the result of a sender.

SENDERS



- A *sender* is an object that describes a unit of async work.
- A *receiver* is an object that handles the result of a sender.
- An *operation state* is an object that represents one instance of an async task.

SENDERS



- A *sender* is an object that describes a unit of async work.
- A *receiver* is an object that handles the result of a sender.
- An *operation state* is an object that represents one instance of an async task.
- A *scheduler* is a handle to an execution context.

SENDERS

ALGORITHMS



Signature

```
auto factory(non_sender...) -> sender
```

```
auto adaptor(sender..., non_sender...) -> sender
```

```
auto consumer(sender..., non_sender...) -> non_sender
```

SENDERS

NOTE ON PIPE OPERATORS

sender | adaptor() ↔ adaptor(sender)

sender | adaptor(value) ↔ adaptor(sender, value)

SENDERS

EXAMPLE

```
1 auto sched = stdexec::get_parallel_scheduler();  
2  
3 auto work = stdexec::just(42)  
4           | stdexec::continues_on(sched)  
5           | stdexec::then([](int value) { return value + 1; });  
6  
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

SENDERS

EXAMPLE

```
1 auto sched = stdexec::get_parallel_scheduler();  
2  
3 auto work = stdexec::just(42)  
4     | stdexec::continues_on(sched)  
5     | stdexec::then([](int value) { return value + 1; });  
6  
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

SENDERS

EXAMPLE

```
1 auto sched = stdexec::get_parallel_scheduler();  
2  
3 auto work = stdexec::just(42)  
4     | stdexec::continues_on(sched)  
5     | stdexec::then([](int value) { return value + 1; });  
6  
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

SENDERS

EXAMPLE

```
1 auto sched = stdexec::get_parallel_scheduler();  
2  
3 auto work = stdexec::just(42)  
4           | stdexec::continues_on(sched)  
5           | stdexec::then([](int value) { return value + 1; });  
6  
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

SENDERS

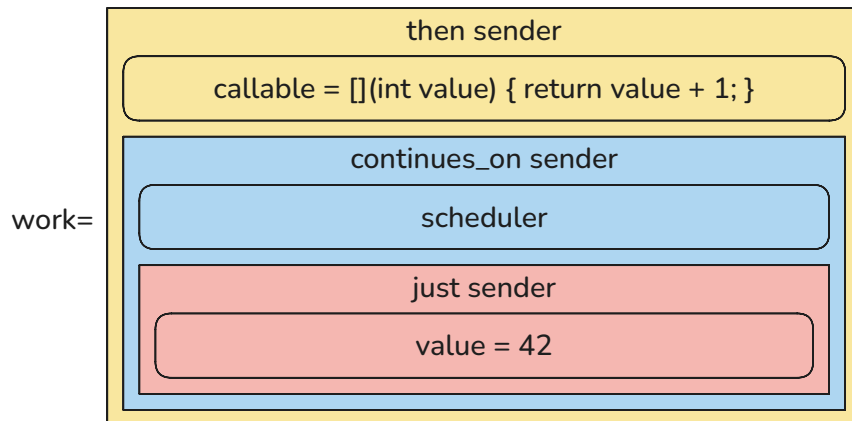
EXAMPLE

```
1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

SENDERS

MEMORY LAYOUT (SENDER)

```
1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

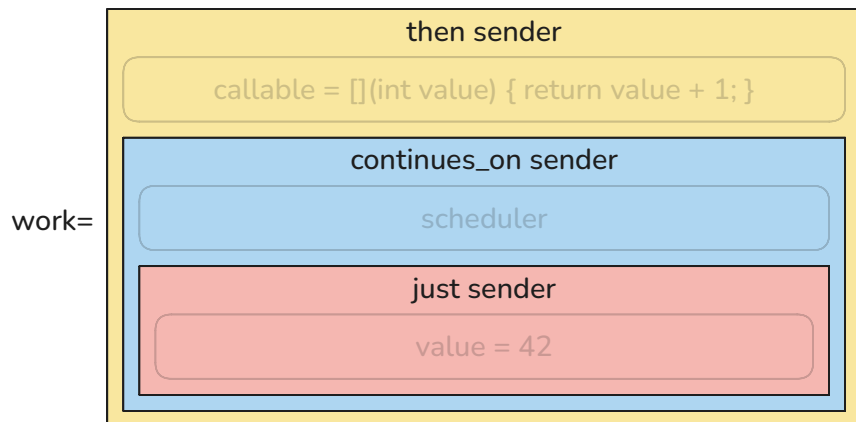


- This is the recipe for the work.

SENDERS

MEMORY LAYOUT (SENDER)

```
1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

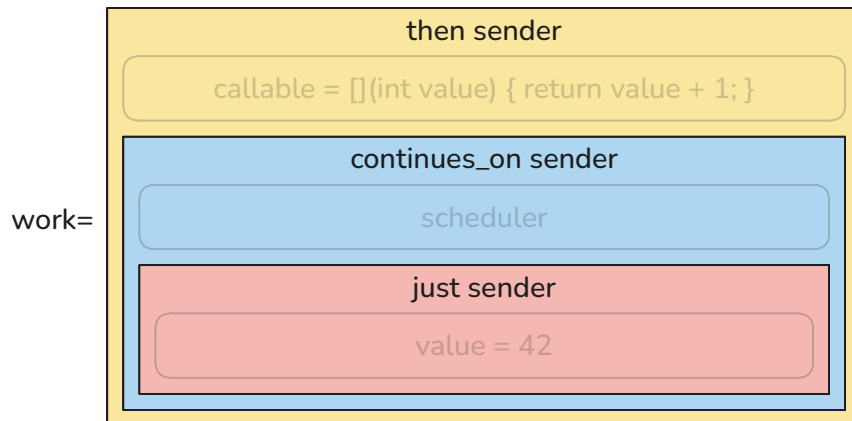


- This is the recipe for the work.
- Parents wrap children in memory.

SENDERS

MEMORY LAYOUT (SENDER)

```
1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

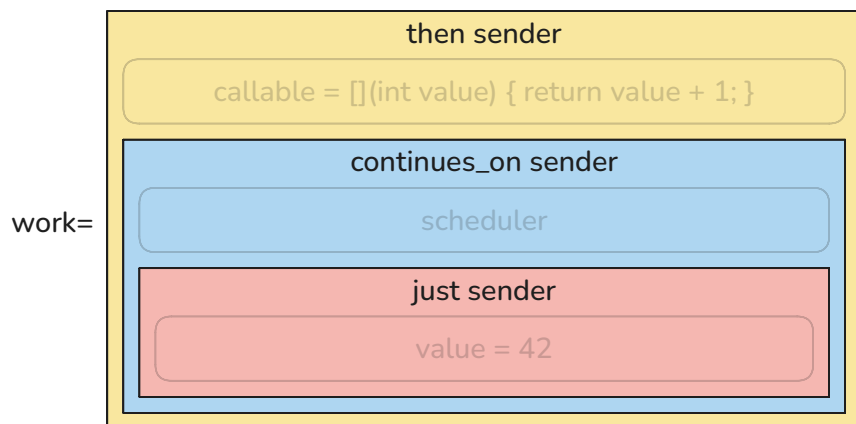


- This is the recipe for the work.
- Parents wrap children in memory.
- Can be passed to other sender adaptors.

SENDERS

MEMORY LAYOUT (SENDER)

```
1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```



- This is the recipe for the work.
- Parents wrap children in memory.
- Can be passed to other sender adaptors.
- Can be connected to a receiver.

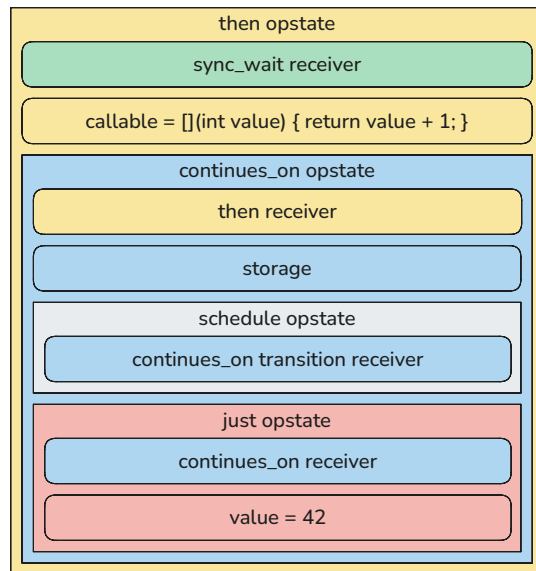
SENDERS

MEMORY LAYOUT (OPERATION STATE)

```

1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4   | stdexec::continues_on(sched)
5   | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();

```



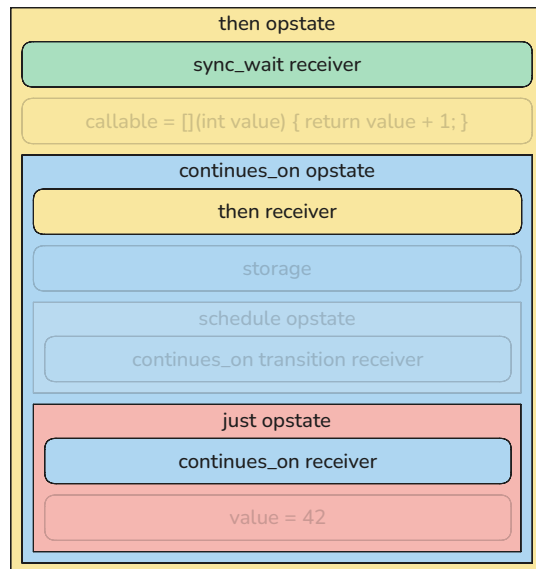
- Holds all the data needed for the operation.

SENDERS

MEMORY LAYOUT (OPERATION STATE)

```

1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
  
```



- Holds all the data needed for the operation.
- Mirrors the sender structure.

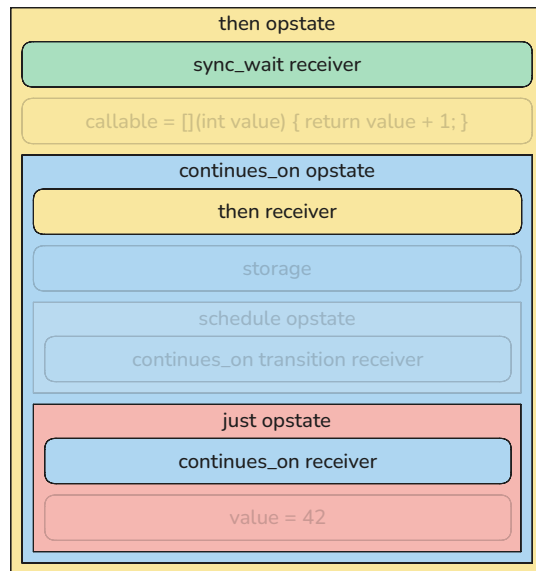
SENDERS

MEMORY LAYOUT (OPERATION STATE)

```

1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();

```



- Holds all the data needed for the operation.
- Mirrors the sender structure.
- Receivers are the glue between layers.

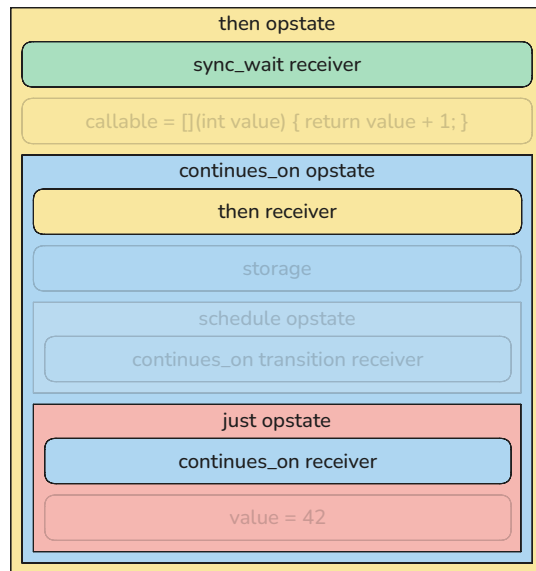
SENDERS

MEMORY LAYOUT (OPERATION STATE)

```

1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4   | stdexec::continues_on(sched)
5   | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();

```



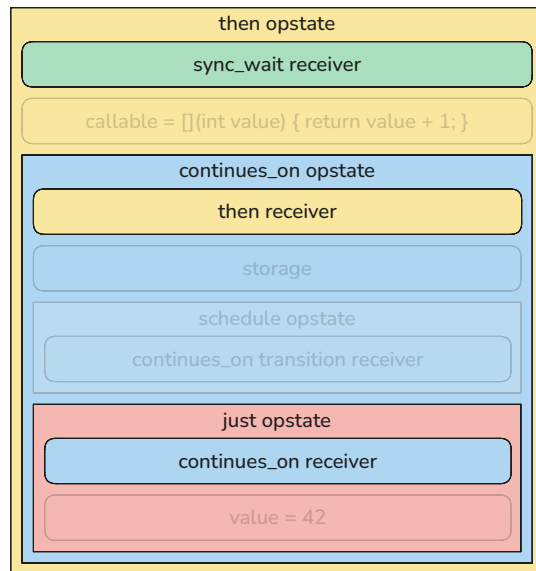
- Holds all the data needed for the operation.
- Mirrors the sender structure.
- Receivers are the glue between layers.
- Runs only when explicitly started.

SENDERS

MEMORY LAYOUT (OPERATION STATE)

```

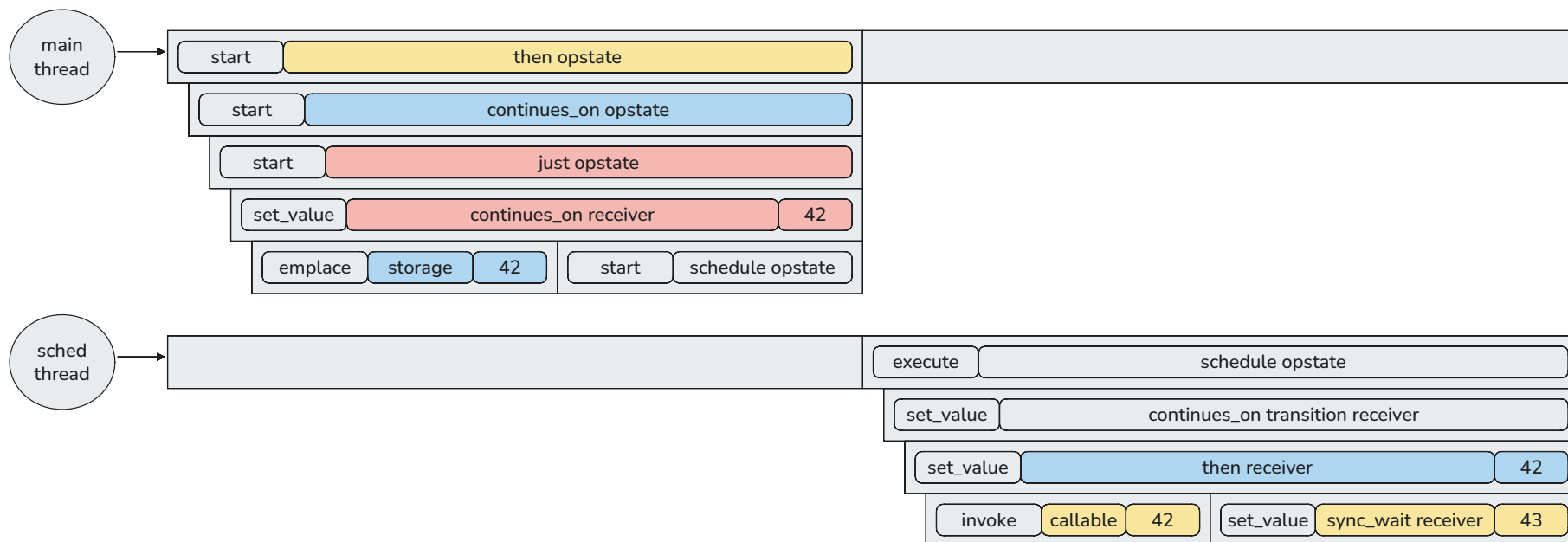
1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
  
```



- Holds all the data needed for the operation.
- Mirrors the sender structure.
- Receivers are the glue between layers.
- Runs only when explicitly started.
- Typically destroyed upon completion.

SENDERS

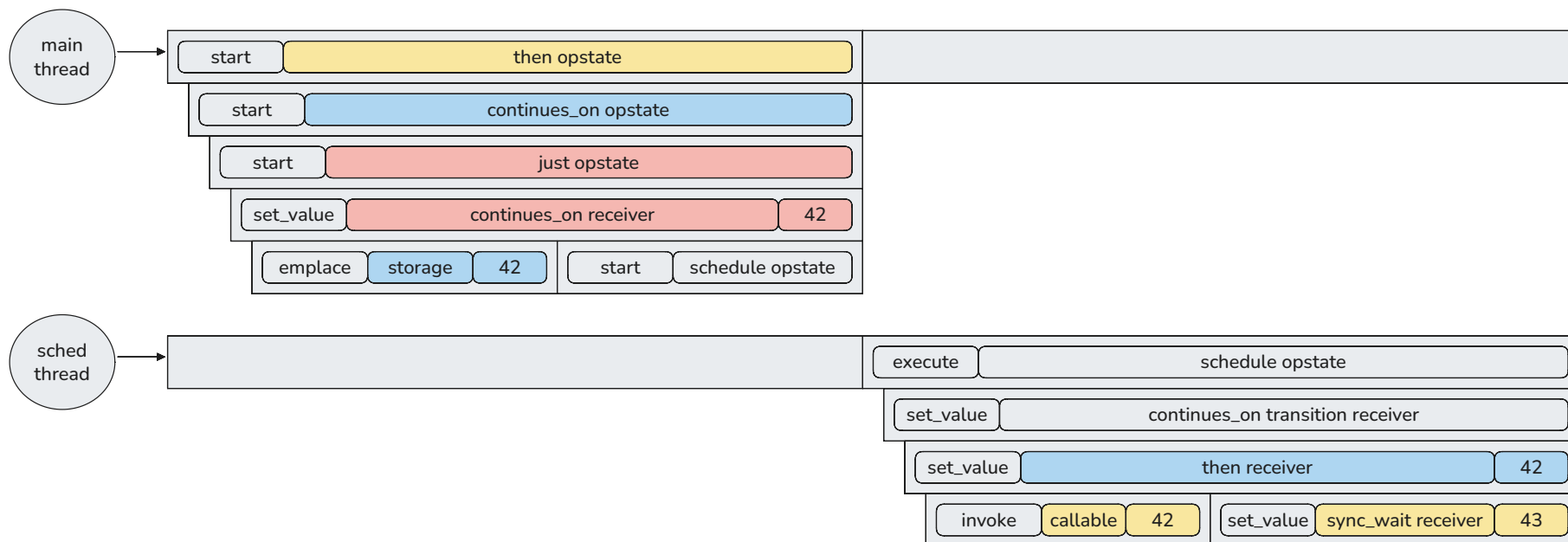
```
1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```



SENDERS

```
1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

the compiler can inline function calls



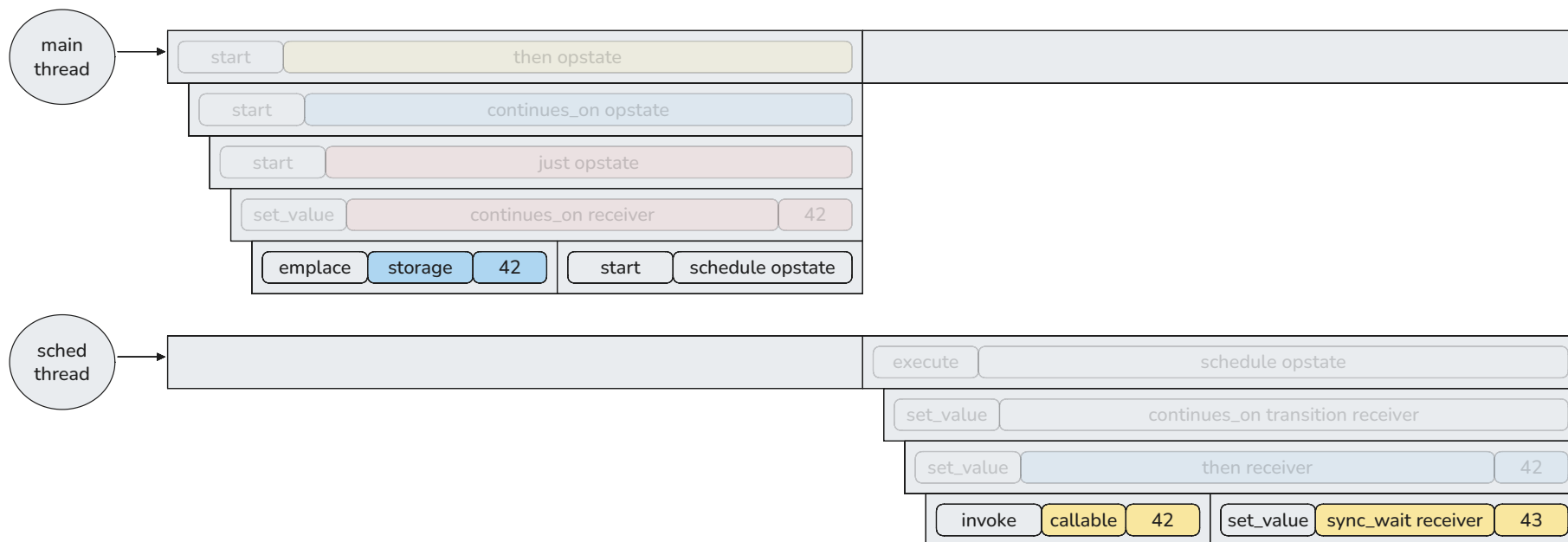
SENDERS

```

1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();

```

store 42 → hop to thread → invoke lambda → complete with 43



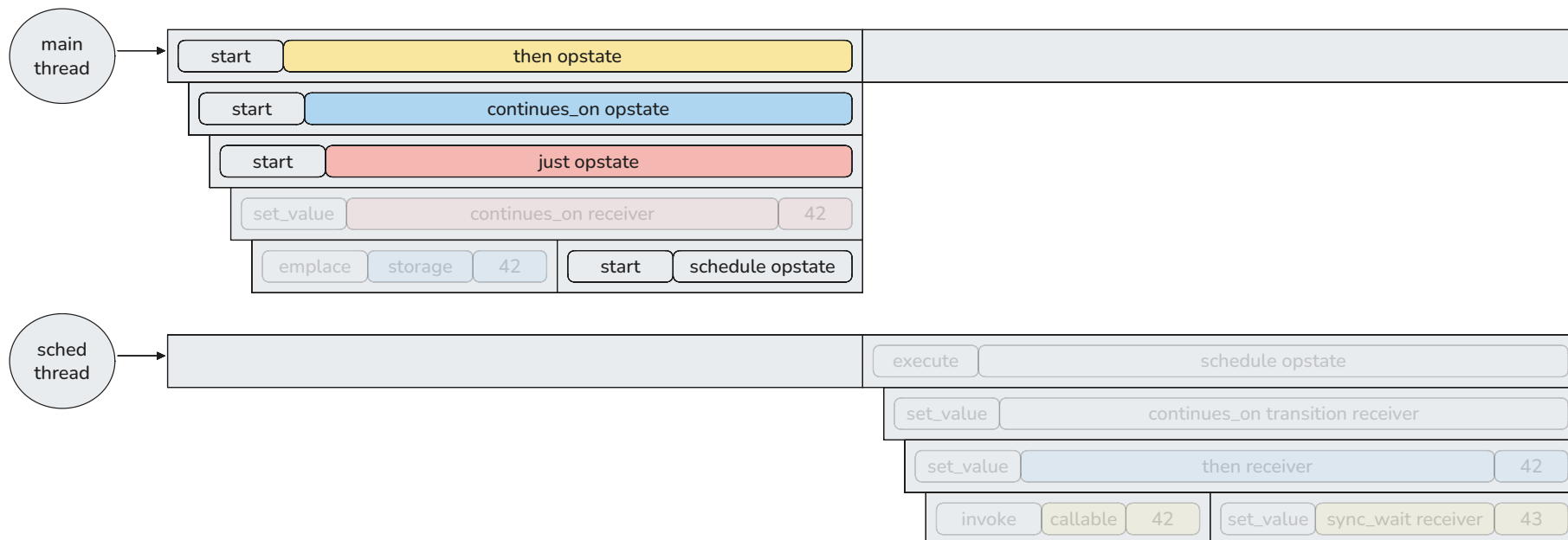
SENDERS

```

1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();

```

start propagates from parent to child



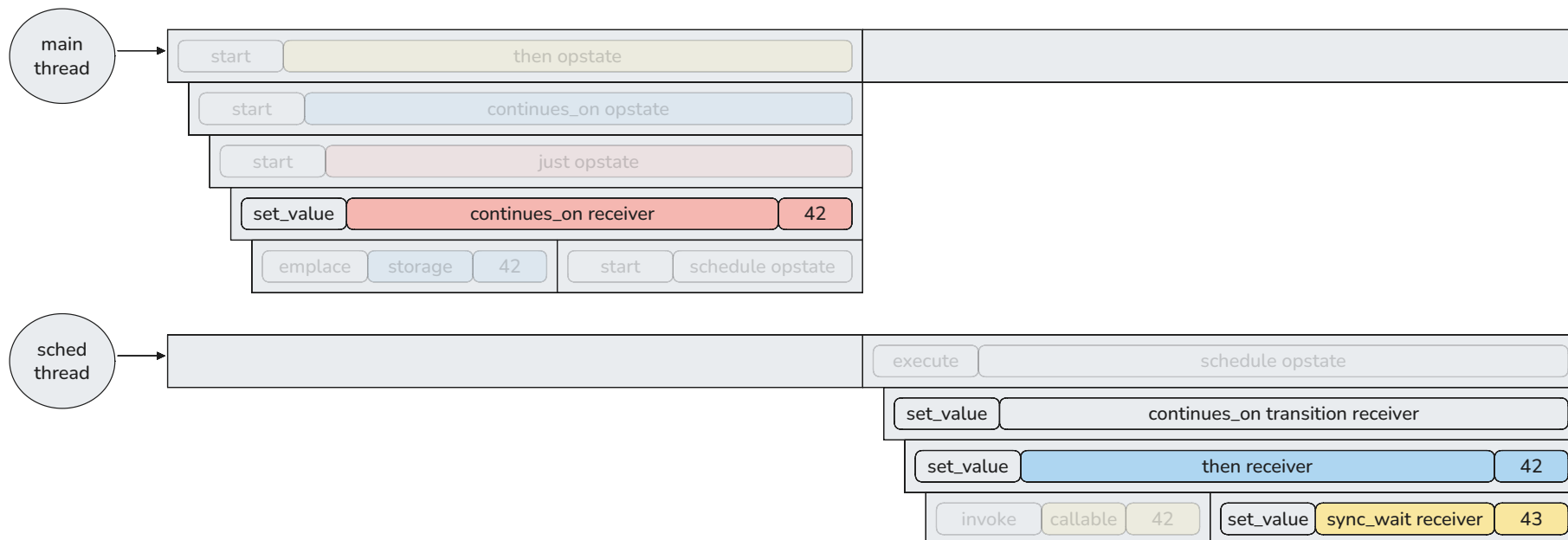
SENDERS

```

1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();

```

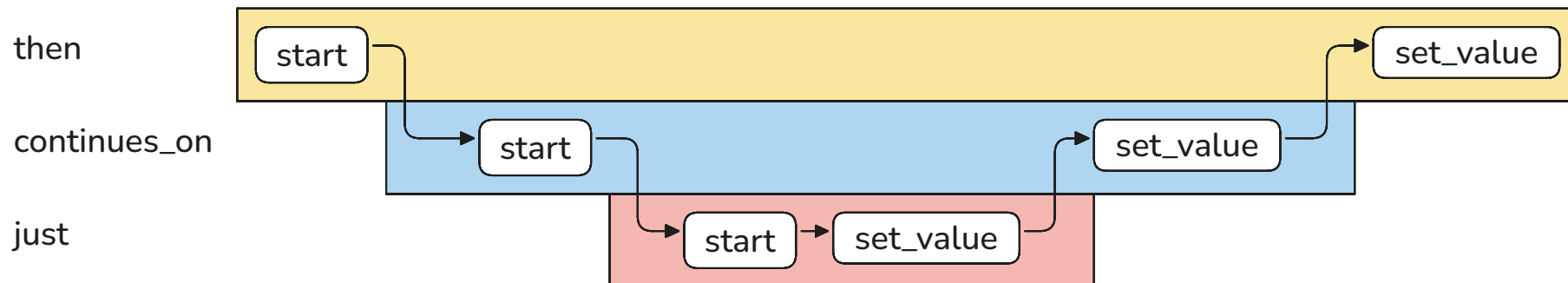
set_value propagates from child to parent



SENDERS

```
1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

execution structure



SENDERS

```
1 auto sched = stdexec::get_parallel_scheduler();
2
3 auto work = stdexec::just(42)
4     | stdexec::continues_on(sched)
5     | stdexec::then([](int value) { return value + 1; });
6
7 auto [value] = stdexec::sync_wait(std::move(work)).value();
```

abstraction

work



SENDERS

Cancellation is cooperative.

```
1 auto token = stdexec::get_stop_token(stdexec::get_env(receiver));  
2  
3 if (token.stop_requested()) { /* complete early */ }  
4  
5 auto stop_callback = stop_callback_type(token, [] { /* complete early */ });
```

SENDERS

Cancellation is cooperative.

```
1 auto token = stdexec::get_stop_token(stdexec::get_env(receiver));  
2  
3 if (token.stop_requested()) { /* complete early */ }  
4  
5 auto stop_callback = stop_callback_type(token, [] { /* complete early */ });
```

- Receivers provide queryable *environments*.

SENDERS

Cancellation is cooperative.

```
1 auto token = stdexec::get_stop_token(stdexec::get_env(receiver));  
2  
3 if (token.stop_requested()) { /* complete early */ }  
4  
5 auto stop_callback = stop_callback_type(token, [] { /* complete early */ });
```

- Receivers provide queryable *environments*.
- Senders can query the receiver *environment* for a stop token at any point.

SENDERS

Cancellation is cooperative.

```
1 auto token = stdexec::get_stop_token(stdexec::get_env(receiver));  
2  
3 if (token.stop_requested()) { /* complete early */ }  
4  
5 auto stop_callback = stop_callback_type(token, [] { /* complete early */ });
```

- Receivers provide queryable *environments*.
- Senders can query the receiver *environment* for a stop token at any point.
- They can check the token directly.

SENDERS

Cancellation is cooperative.

```
1 auto token = stdexec::get_stop_token(stdexec::get_env(receiver));  
2  
3 if (token.stop_requested()) { /* complete early */ }  
4  
5 auto stop_callback = stop_callback_type(token, [] { /* complete early */ });
```

- Receivers provide queryable *environments*.
- Senders can query the receiver *environment* for a stop token at any point.
- They can check the token directly.
- They can register a callback.

SENDERS

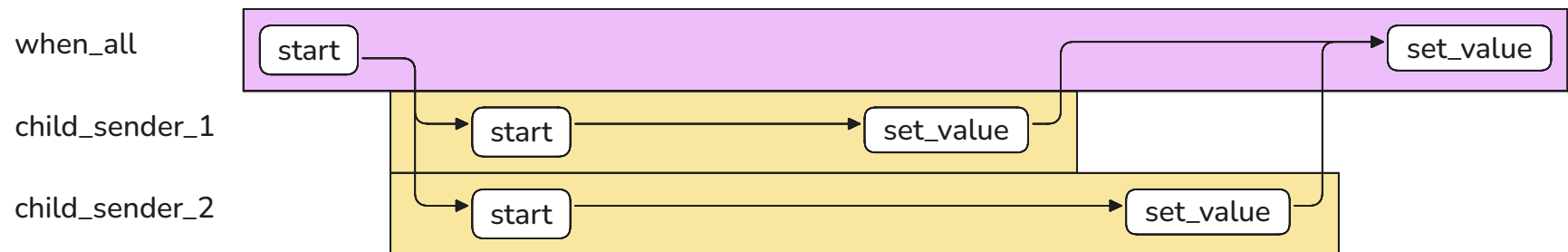
ALGORITHMS

```
1 auto work = stdexec::when_all(child_sender_1, child_sender_2);
```

SENDERS

ALGORITHMS

```
1 auto work = stdexec::when_all(child_sender_1, child_sender_2);
```

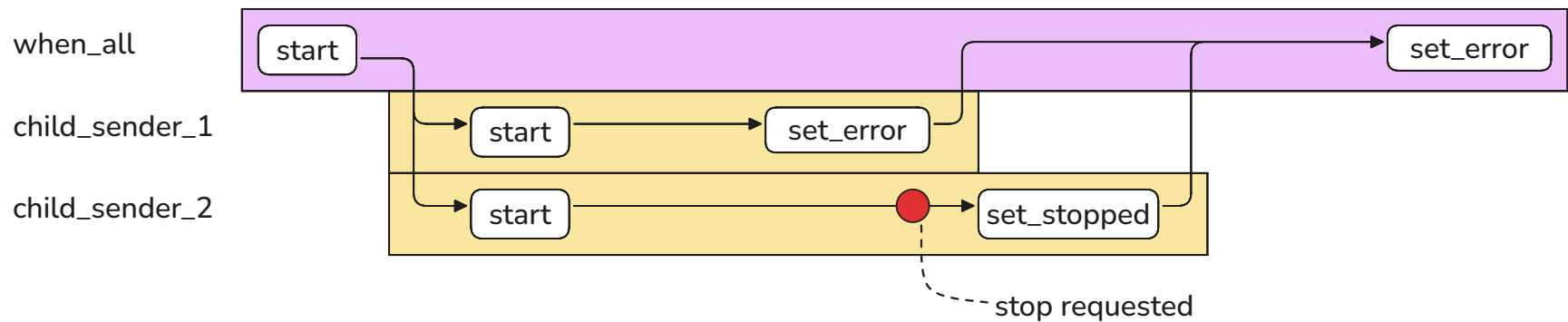


- Completes when all children complete.

SENDERS

ALGORITHMS

```
1 auto work = stdexec::when_all(child_sender_1, child_sender_2);
```



- Completes when all children complete.
- If a child fails, it stops the others.

SENDERS

ALGORITHMS

```
1 auto work = child_sender  
2   | stdexec::let_value([](auto value) { return more_work(value); });
```

SENDERS

ALGORITHMS

```
1 auto work = child_sender  
2   | stdexec::let_value([](auto value) { return more_work(value); });
```

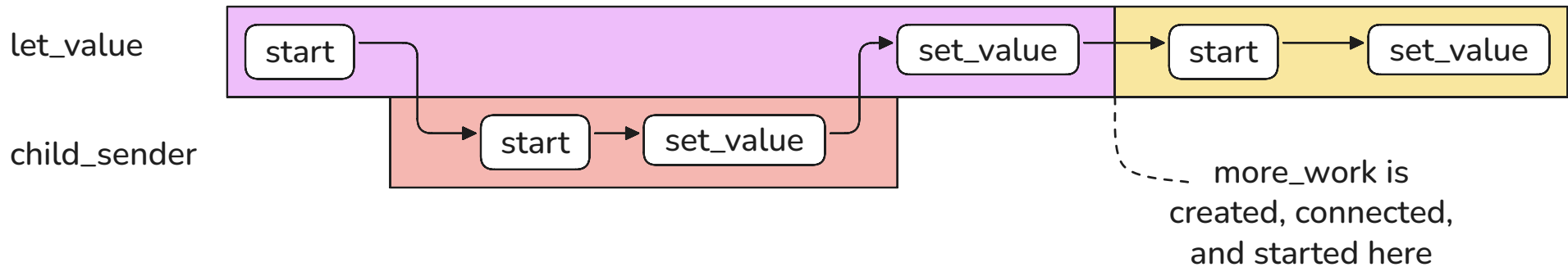
- Similar to the then sender but the callable returns a sender.

SENDERS

ALGORITHMS

```
1 auto work = child_sender
2   | stdexec::let_value([](auto value) { return more_work(value); });
```

- Similar to the then sender but the callable returns a sender.



SENDERS

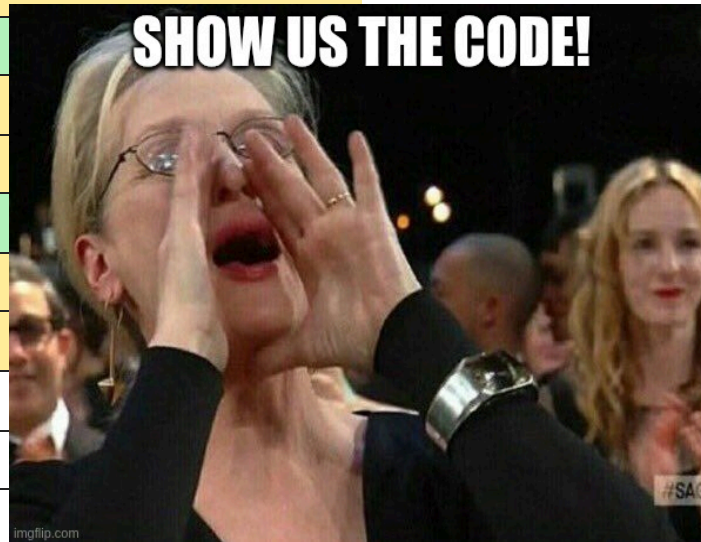
ALGORITHMS

<code>std::execution::just</code>	<code>std::execution::read_env</code>
<code>std::execution::just_error</code>	<code>std::execution::write_env</code>
<code>std::execution::just_stopped</code>	<code>std::execution::bulk</code>
<code>std::execution::then</code>	<code>std::execution::bulk_chunked</code>
<code>std::execution::upon_error</code>	<code>std::execution::bulk_unchunked</code>
<code>std::execution::upon_stopped</code>	<code>std::execution::when_all</code>
<code>std::execution::let_value</code>	<code>std::execution::when_all_with_variant</code>
<code>std::execution::let_error</code>	<code>std::execution::into_variant</code>
<code>std::execution::let_stopped</code>	<code>std::execution::stopped_as_optional</code>
<code>std::execution::unstoppable</code>	<code>std::execution::stopped_as_error</code>
<code>std::execution::schedule</code>	<code>std::execution::associate</code>
<code>std::execution::schedule_from</code>	<code>std::execution::spawn_future</code>
<code>std::execution::starts_on</code>	<code>std::this_thread::sync_wait</code>
<code>std::execution::continues_on</code>	<code>std::this_thread::sync_wait_with_variant</code>
<code>std::execution::on</code>	<code>std::execution::spawn</code>

SENDERS

ALGORITHMS

std::execution::just	std::execution::read_env
std::execution::just_error	std::execution::write_env
std::execution::just_stopped	std::execution::bulk
std::execution::then	std::execution::bulk_chunked
std::execution::upon_error	std::execution::bulk_unchunked
std::execution::upon_stopped	std::execution::then_all
std::execution::let_value	std::execution::then_all_with_variant
std::execution::let_error	std::execution::into_variant
std::execution::let_stopped	std::execution::stopped_as_optional
std::execution::unstoppable	std::execution::stopped_as_error
std::execution::schedule	std::execution::associate
std::execution::schedule_from	std::execution::spawn_future
std::execution::starts_on	std::this_thread::sync_wait
std::execution::continues_on	std::this_thread::sync_wait_with_variant
std::execution::on	std::execution::spawn



ADAPTER LAYER

VERSION COMPATIBILITY

- C++20
- stdexec
- ROS 2 Lyrical



ADAPTER LAYER

`rclexec::execute_action`

ROS Action Client API	Completion
<code>async_send_goal</code>	• <code>set_value</code> when the goal is achieved
<code>async_get_result</code>	→ • <code>set_error</code> when the goal is rejected or aborted
<code>async_cancel_goal</code>	• <code>set_stopped</code> when the goal is canceled

ADAPTER LAYER

`rclexec::execute_action`

ROS Action Client API

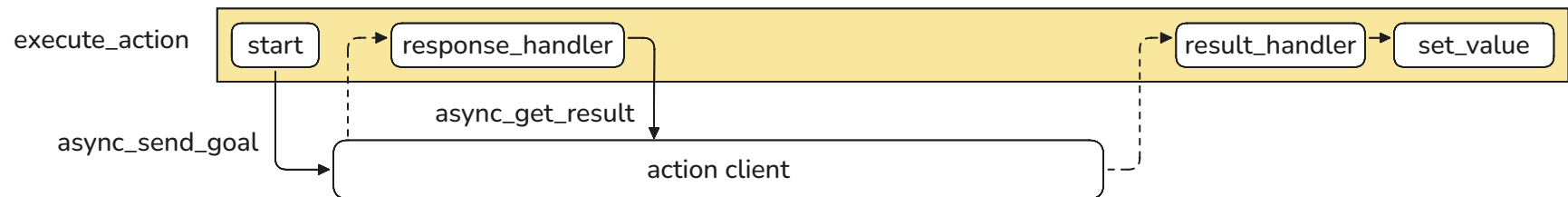
Completion

`async_send_goal`

`async_get_result`

`async_cancel_goal`

- `set_value` when the goal is achieved
- `set_error` when the goal is rejected or aborted
- `set_stopped` when the goal is canceled



ADAPTER LAYER

`rclexec::execute_action`

ROS Action Client API

Completion

`async_send_goal`

`async_get_result`

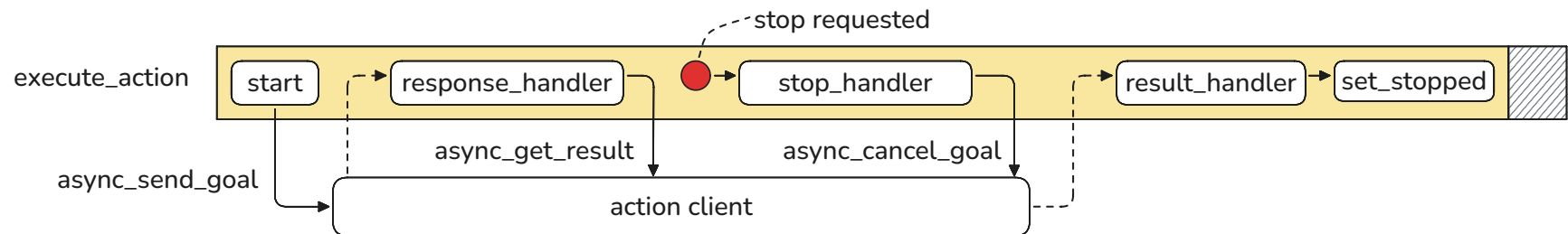
`async_cancel_goal`

- `set_value` when the goal is achieved

→

- `set_error` when the goal is rejected or aborted

- `set_stopped` when the goal is canceled



ADAPTER LAYER

rclexec::execute_action_sender

```
1 template <typename T>
2 class execute_action_sender {
3     private:
4         template <typename R>
5         class opstate;
6
7     public:
8         using sender_concept = stdexec::sender_tag;
9
10        using client_type = rclcpp_action::Client<T>;
11        using goal_handle_type = rclcpp_action::ClientGoalHandle<T>;
12        using goal_type = typename T::Goal;
13        using result_type = typename T::Result;
14
15        using completion_signatures = stdexec::completion_signatures<
16            stdexec::set_value_t(std::shared_ptr<result_type>),
17            stdexec::set_error_t(std::exception_ptr),
18            stdexec::set_stopped_t()>;
19
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

1  template <typename T>
2  class execute_action_sender {
3  private:
4      template <typename R>
5      class opstate;
6
7  public:
8      using sender_concept = stdexec::sender_tag;
9
10     using client_type = rclcpp_action::Client<T>;
11     using goal_handle_type = rclcpp_action::ClientGoalHandle<T>;
12     using goal_type = typename T::Goal;
13     using result_type = typename T::Result;
14
15     using completion_signatures = stdexec::completion_signatures<
16         stdexec::set_value_t(std::shared_ptr<result_type>),
17         stdexec::set_error_t(std::exception_ptr),
18         stdexec::set_stopped_t()>;
19

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

1  template <typename T>
2  class execute_action_sender {
3  private:
4      template <typename R>
5      class opstate;
6
7  public:
8      using sender_concept = stdexec::sender_tag;
9
10     using client_type = rclcpp_action::Client<T>;
11     using goal_handle_type = rclcpp_action::ClientGoalHandle<T>;
12     using goal_type = typename T::Goal;
13     using result_type = typename T::Result;
14
15     using completion_signatures = stdexec::completion_signatures<
16         stdexec::set_value_t(std::shared_ptr<result_type>),
17         stdexec::set_error_t(std::exception_ptr),
18         stdexec::set_stopped_t()>;
19

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

2  class execute_action_sender {
3  private:
4      template <typename R>
5      class opstate;
6
7  public:
8      using sender_concept = stdexec::sender_tag;
9
10     using client_type = rclcpp_action::Client<T>;
11     using goal_handle_type = rclcpp_action::ClientGoalHandle<T>;
12     using goal_type = typename T::Goal;
13     using result_type = typename T::Result;
14
15     using completion_signatures = stdexec::completion_signatures<
16         stdexec::set_value_t(std::shared_ptr<result_type>),
17         stdexec::set_error_t(std::exception_ptr),
18         stdexec::set_stopped_t()>;
19
20     execute_action_sender(std::shared_ptr<client_type> client, goal_type goal)
21         : client_(std::move(client)) {}

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

7 public:
8     using sender_concept = stdexec::sender_tag;
9
10    using client_type = rclcpp_action::Client<T>;
11    using goal_handle_type = rclcpp_action::ClientGoalHandle<T>;
12    using goal_type = typename T::Goal;
13    using result_type = typename T::Result;
14
15    using completion_signatures = stdexec::completion_signatures<
16        stdexec::set_value_t(std::shared_ptr<result_type>),
17        stdexec::set_error_t(std::exception_ptr),
18        stdexec::set_stopped_t()>;
19
20    execute_action_sender(std::shared_ptr<client_type> client, goal_type goal)
21        : client_(std::move(client)),
22          goal_(std::move(goal)) {}
23
24    template <typename R>
25    auto connect(R receiver) const noexcept -> opstate<R>;
26

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

15 using completion_signatures = stdexec::completion_signatures<
16     stdexec::set_value_t(std::shared_ptr<result_type>),
17     stdexec::set_error_t(std::exception_ptr),
18     stdexec::set_stopped_t()>;
19
20 execute_action_sender(std::shared_ptr<client_type> client, goal_type goal)
21     : client_(std::move(client)),
22       goal_(std::move(goal)) {}
23
24 template <typename R>
25 auto connect(R receiver) const noexcept -> opstate<R>;
26
27 private:
28     std::shared_ptr<client_type> client_;
29     goal_type goal_;
30 };
31
32 template <typename T>
33 template <typename R>
34 class execute_action_sender<T>: opstate<R> {

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

15 using completion_signatures = stdexec::completion_signatures<
16     stdexec::set_value_t(std::shared_ptr<result_type>),
17     stdexec::set_error_t(std::exception_ptr),
18     stdexec::set_stopped_t()>;
19
20 execute_action_sender(std::shared_ptr<client_type> client, goal_type goal)
21     : client_(std::move(client)),
22     goal_(std::move(goal)) {}
23
24 template <typename R>
25 auto connect(R receiver) const noexcept -> opstate<R>;
26
27 private:
28     std::shared_ptr<client_type> client_;
29     goal_type goal_;
30 };
31
32 template <typename T>
33 template <typename R>
34 class execute_action_sender<T>::opstate {

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

131         std::move(self->receiver_),
132         std::make_exception_ptr(action_unknown_result_code("unknown error code")));
133     break;
134 }
135 }
136 };
137
138 template <typename T>
139 template <typename R>
140 auto execute_action_sender<T>::connect(R receiver) const noexcept -> opstate<R> {
141     return opstate<R>{client_, goal_, std::move(receiver)};
142 }
143
144 struct empty_optional {
145     constexpr void emplace(auto&&...) noexcept {}
146     constexpr void reset() noexcept {}
147     constexpr bool has_value() const noexcept { return false; }
148     constexpr explicit operator bool() const noexcept { return false; }
149 };

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```
24  template <typename R>
25  auto connect(R receiver) const noexcept -> opstate<R>;
26
27  private:
28      std::shared_ptr<client_type> client_;
29      goal_type goal_;
30  };
31
32  template <typename T>
33  template <typename R>
34  class execute_action_sender<T>::opstate {
35  public:
36      using operation_state_concept = stdexec::operation_state_tag;
37
38      opstate(std::shared_ptr<client_type> client, goal_type goal, R receiver)
39          : client_(std::move(client)),
40            goal_(std::move(goal)),
41            receiver_(std::move(receiver)) {}
42
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```
27 private:
28     std::shared_ptr<client_type> client_;
29     goal_type goal_;
30 };
31
32 template <typename T>
33 template <typename R>
34 class execute_action_sender<T>::opstate {
35 public:
36     using operation_state_concept = stdexec::operation_state_tag;
37
38     opstate(std::shared_ptr<client_type> client, goal_type goal, R receiver)
39         : client_(std::move(client)),
40           goal_(std::move(goal)),
41           receiver_(std::move(receiver)) {}
42
43     void start() & noexcept {
44         auto token = stdexec::get_stop_token(stdexec::get_env(receiver_));
45     }
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

30 },
31
32 template <typename T>
33 template <typename R>
34 class execute_action_sender<T>::opstate {
35 public:
36     using operation_state_concept = stdexec::operation_state_tag;
37
38     opstate(std::shared_ptr<client_type> client, goal_type goal, R receiver)
39         : client_(std::move(client)),
40           goal_(std::move(goal)),
41           receiver_(std::move(receiver)) {}
42
43     void start() & noexcept {
44         auto token = stdexec::get_stop_token(stdexec::get_env(receiver_));
45
46         if (token.stop_requested()) {
47             stdexec::set_stopped(std::move(receiver_));
48             return;
49         }

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```
70
71 using stop_token = stdexec::stop_token_of_t<stdexec::env_of_t<R>>;
72 using stop_callback = stdexec::stop_callback_for_t<stop_token, stop_handler>;
73 using stop_callback_storage = std::conditional_t<
74     stdexec::unstoppable_token<stop_token>,
75     empty_optional,
76     std::optional<stop_callback>>;
77
78 std::shared_ptr<client_type> client_;
79 goal_type goal_;
80 [[no_unique_address]] R receiver_;
81 [[no_unique_address]] stop_callback_storage stop_callback_;
82 };
83
84 template <typename T>
85 template <typename R>
86 struct execute_action_sender<T>::opstate<R>::response_handler {
87     opstate* self;
88 }
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

72  using stop_callback = stdexec::stop_callback_for_t<stop_token, stop_handler>;
73  using stop_callback_storage = std::conditional_t<
74      stdexec::unstoppable_token<stop_token>,
75      empty_optional,
76      std::optional<stop_callback>>;
77
78  std::shared_ptr<client_type> client_;
79  goal_type goal_;
80  [[no_unique_address]] R receiver_;
81  [[no_unique_address]] stop_callback_storage stop_callback_;
82  };
83
84  template <typename T>
85  template <typename R>
86  struct execute_action_sender<T>::opstate<R>::response_handler {
87      opstate* self;
88
89      void operator()(std::shared_ptr<goal_handle_type> handle) noexcept {
90          if (!handle) {

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

65     std::shared_ptr<client_type> client_;
66     std::shared_ptr<goal_handle_type> handle;
67
68     void operator()() noexcept { client->async_cancel_goal(handle); }
69 };
70
71 using stop_token = stdexec::stop_token_of_t<stdexec::env_of_t<R>>;
72 using stop_callback = stdexec::stop_callback_for_t<stop_token, stop_handler>;
73 using stop_callback_storage = std::conditional_t<
74     stdexec::unstoppable_token<stop_token>,
75     empty_optional,
76     std::optional<stop_callback>>;
77
78 std::shared_ptr<client_type> client_;
79 goal_type goal_;
80 [[no_unique_address]] R receiver_;
81 [[no_unique_address]] stop_callback_storage stop_callback_;
82 };
83
84 template <typename T>

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

65     std::shared_ptr<client_type> client;
66     std::shared_ptr<goal_handle_type> handle;
67
68     void operator()() noexcept { client->async_cancel_goal(handle); }
69 };
70
71     using stop_token = stdexec::stop_token_of_t<stdexec::env_of_t<R>>;
72     using stop_callback = stdexec::stop_callback_for_t<stop_token, stop_handler>;
73     using stop_callback_storage = std::conditional_t<
74         stdexec::unstoppable_token<stop_token>,
75         empty_optional,
76         std::optional<stop_callback>>;
77
78     std::shared_ptr<client_type> client_;
79     goal_type goal_;
80     [[no_unique_address]] R receiver_;
81     [[no_unique_address]] stop_callback_storage stop_callback_;
82 };
83

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

66     std::shared_ptr<goal_handle_type> handle;
67
68     void operator()() noexcept { client->async_cancel_goal(handle); }
69 };
70
71     using stop_token = stdexec::stop_token_of_t<stdexec::env_of_t<R>>;
72     using stop_callback = stdexec::stop_callback_for_t<stop_token, stop_handler>;
73     using stop_callback_storage = std::conditional_t<
74         stdexec::unstoppable_token<stop_token>,
75         empty_optional,
76         std::optional<stop_callback>>;
77
78     std::shared_ptr<client_type> client_;
79     goal_type goal_;
80     [[no_unique_address]] R receiver_;
81     [[no_unique_address]] stop_callback_storage stop_callback_;
82 };
83
84 template <typename T>

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

131         std::move(sett->receiver_),
132         std::make_exception_ptr(action_unknown_result_code("unknown error code")));
133     break;
134 }
135 }
136 };
137
138 template <typename T>
139 template <typename R>
140 auto execute_action_sender<T>::connect(R receiver) const noexcept -> opstate<R> {
141     return opstate<R>{client_, goal_, std::move(receiver)};
142 }
143
144 struct empty_optional {
145     constexpr void emplace(auto&&...) noexcept {}
146     constexpr void reset() noexcept {}
147     constexpr bool has_value() const noexcept { return false; }
148     constexpr explicit operator bool() const noexcept { return false; }
149 };

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

66     std::shared_ptr<goal_handle_type> handle;
67
68     void operator()() noexcept { client->async_cancel_goal(handle); }
69 };
70
71     using stop_token = stdexec::stop_token_of_t<stdexec::env_of_t<R>>;
72     using stop_callback = stdexec::stop_callback_for_t<stop_token, stop_handler>;
73     using stop_callback_storage = std::conditional_t<
74         stdexec::unstoppable_token<stop_token>,
75         empty_optional,
76         std::optional<stop_callback>>;
77
78     std::shared_ptr<client_type> client_;
79     goal_type goal_;
80     [[no_unique_address]] R receiver_;
81     [[no_unique_address]] stop_callback_storage stop_callback_;
82 };
83
84     template <typename T>

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

67
68     void operator()() noexcept { client->async_cancel_goal(handle); }
69 };
70
71     using stop_token = stdexec::stop_token_of_t<stdexec::env_of_t<R>>;
72     using stop_callback = stdexec::stop_callback_for_t<stop_token, stop_handler>;
73     using stop_callback_storage = std::conditional_t<
74         stdexec::unstoppable_token<stop_token>,
75         empty_optional,
76         std::optional<stop_callback>>;
77
78     std::shared_ptr<client_type> client_;
79     goal_type goal_;
80     [[no_unique_address]] R receiver_;
81     [[no_unique_address]] stop_callback_storage stop_callback_;
82 };
83
84 template <typename T>
85 template <typename R>

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

63
64 struct stop_handler {
65     std::shared_ptr<client_type> client;
66     std::shared_ptr<goal_handle_type> handle;
67
68     void operator()() noexcept { client->async_cancel_goal(handle); }
69 };
70
71 using stop_token = stdexec::stop_token_of_t<stdexec::env_of_t<R>>;
72 using stop_callback = stdexec::stop_callback_for_t<stop_token, stop_handler>;
73 using stop_callback_storage = std::conditional_t<
74     stdexec::unstoppable_token<stop_token>,
75     empty_optional,
76     std::optional<stop_callback>>>;
77
78 std::shared_ptr<client_type> client_;
79 goal_type goal_;
80 [[no_unique_address]] R receiver_;
81 [[no_unique_address]] stop_callback_storage stop_callback_;
  
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

62 struct result_handler;
63
64 struct stop_handler {
65     std::shared_ptr<client_type> client;
66     std::shared_ptr<goal_handle_type> handle;
67
68     void operator()() noexcept { client->async_cancel_goal(handle); }
69 };
70
71 using stop_token = stdexec::stop_token_of_t<stdexec::env_of_t<R>>;
72 using stop_callback = stdexec::stop_callback_for_t<stop_token, stop_handler>;
73 using stop_callback_storage = std::conditional_t<
74     stdexec::unstoppable_token<stop_token>,
75     empty_optional,
76     std::optional<stop_callback>>;
77
78 std::shared_ptr<client_type> client_;
79 goal_type goal_;
80 [[no_unique_address]] R receiver_;
  
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

34 class execute_action_sender<T>::opstate {
35 public:
36     using operation_state_concept = stdexec::operation_state_tag;
37
38     opstate(std::shared_ptr<client_type> client, goal_type goal, R receiver)
39         : client_(std::move(client)),
40           goal_(std::move(goal)),
41           receiver_(std::move(receiver)) {}
42
43     void start() & noexcept {
44         auto token = stdexec::get_stop_token(stdexec::get_env(receiver_));
45
46         if (token.stop_requested()) {
47             stdexec::set_stopped(std::move(receiver_));
48             return;
49         }
50
51         typename rclcpp_action::Client<T>::SendGoalOptions options;
52         options.goal_response_callback = response_handler{this};
  
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```
35 public:
36     using operation_state_concept = stdexec::operation_state_tag;
37
38     opstate(std::shared_ptr<client_type> client, goal_type goal, R receiver)
39         : client_(std::move(client)),
40           goal_(std::move(goal)),
41           receiver_(std::move(receiver)) {}
42
43     void start() & noexcept {
44         auto token = stdexec::get_stop_token(stdexec::get_env(receiver_));
45
46         if (token.stop_requested()) {
47             stdexec::set_stopped(std::move(receiver_));
48             return;
49         }
50
51         typename rclcpp_action::Client<T>::SendGoalOptions options;
52         options.goal_response_callback = response_handler{this};
53         try {
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

38 opstate(std::shared_ptr<client_type> client, goal_type goal, R receiver)
39 : client_(std::move(client)),
40   goal_(std::move(goal)),
41   receiver_(std::move(receiver)) {}
42
43 void start() & noexcept {
44     auto token = stdexec::get_stop_token(stdexec::get_env(receiver_));
45
46     if (token.stop_requested()) {
47         stdexec::set_stopped(std::move(receiver_));
48         return;
49     }
50
51     typename rclcpp_action::Client<T>::SendGoalOptions options;
52     options.goal_response_callback = response_handler{this};
53     try {
54         client_>async_send_goal(std::move(goal_), std::move(options));
55     } catch (...) {
56         stdexec::set_error(std::move(receiver_), std::current_exception());
57     }

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

42
43 void start() & noexcept {
44     auto token = stdexec::get_stop_token(stdexec::get_env(receiver_));
45
46     if (token.stop_requested()) {
47         stdexec::set_stopped(std::move(receiver_));
48         return;
49     }
50
51     typename rclcpp_action::Client<T>::SendGoalOptions options;
52     options.goal_response_callback = response_handler{this};
53     try {
54         client_>async_send_goal(std::move(goal_), std::move(options));
55     } catch (...) {
56         stdexec::set_error(std::move(receiver_), std::current_exception());
57     }
58 }
59
60 private:
61     struct response_handler

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```
46  if (token.stop_requested()) {
47      stdexec::set_stopped(std::move(receiver_));
48      return;
49  }
50
51  typename rclcpp_action::Client<T>::SendGoalOptions options;
52  options.goal_response_callback = response_handler{this};
53  try {
54      client_->async_send_goal(std::move(goal_), std::move(options));
55  } catch (...) {
56      stdexec::set_error(std::move(receiver_), std::current_exception());
57  }
58  }
59
60  private:
61      struct response_handler;
62      struct result_handler;
63
64      struct stop_handler {
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```
76     std::optional<stop_callback>;
77
78     std::shared_ptr<client_type> client_;
79     goal_type goal_;
80     [[no_unique_address]] R receiver_;
81     [[no_unique_address]] stop_callback_storage stop_callback_;
82 };
83
84 template <typename T>
85 template <typename R>
86 struct execute_action_sender<T>::opstate<R>::response_handler {
87     opstate* self;
88
89     void operator()(std::shared_ptr<goal_handle_type> handle) noexcept {
90         if (!handle) {
91             stdexec::set_error(
92                 std::move(self->receiver_),
93                 std::make_exception_ptr(action_rejected("goal rejected by the server")));
94         }
95     }
96 }
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```
78  std::shared_ptr<client_type> client_;
79  goal_type goal_;
80  [[no_unique_address]] R receiver_;
81  [[no_unique_address]] stop_callback_storage stop_callback_;
82 };
83
84 template <typename T>
85 template <typename R>
86 struct execute_action_sender<T>::opstate<R>::response_handler {
87     opstate* self;
88
89     void operator()(std::shared_ptr<goal_handle_type> handle) noexcept {
90         if (!handle) {
91             stdexec::set_error(
92                 std::move(self->receiver_),
93                 std::make_exception_ptr(action_rejected("goal rejected by the server")));
94             return;
95         }
96     }
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

80  [[no_unique_address]] R receiver_;
81  [[no_unique_address]] stop_callback_storage stop_callback_;
82  };
83
84  template <typename T>
85  template <typename R>
86  struct execute_action_sender<T>::opstate<R>::response_handler {
87      opstate* self;
88
89      void operator()(std::shared_ptr<goal_handle_type> handle) noexcept {
90          if (!handle) {
91              stdexec::set_error(
92                  std::move(self->receiver_),
93                  std::make_exception_ptr(action_rejected("goal rejected by the server")));
94              return;
95          }
96
97          auto token = stdexec::get_stop_token(stdexec::get_env(self->receiver_));
98          self->stop_callback_.emplace(token, stop_handler{self->client_, handle});

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

83
84 template <typename T>
85 template <typename R>
86 struct execute_action_sender<T>::opstate<R>::response_handler {
87     opstate* self;
88
89     void operator()(std::shared_ptr<goal_handle_type> handle) noexcept {
90         if (!handle) {
91             stdexec::set_error(
92                 std::move(self->receiver_),
93                 std::make_exception_ptr(action_rejected("goal rejected by the server")));
94             return;
95         }
96
97         auto token = stdexec::get_stop_token(stdexec::get_env(self->receiver_));
98         self->stop_callback_.emplace(token, stop_handler{self->client_, handle});
99
100         try {
101             self->client_->async_get_result(handle, result_handler{self});
102         } catch (...) {

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

88
89 void operator()(std::shared_ptr<goal_handle_type> handle) noexcept {
90     if (!handle) {
91         stdexec::set_error(
92             std::move(self->receiver_),
93             std::make_exception_ptr(action_rejected("goal rejected by the server")));
94         return;
95     }
96
97     auto token = stdexec::get_stop_token(stdexec::get_env(self->receiver_));
98     self->stop_callback_.emplace(token, stop_handler{self->client_, handle});
99
100     try {
101         self->client_->async_get_result(handle, result_handler{self});
102     } catch (...) {
103         self->stop_callback_->reset();
104         stdexec::set_error(std::move(self->receiver_), std::current_exception());
105     }
106 }
107

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

93     std::make_exception_ptr(action_rejected{ "got rejected by the server" });
94     return;
95 }
96
97 auto token = stdexec::get_stop_token(stdexec::get_env(self->receiver_));
98 self->stop_callback_.emplace(token, stop_handler{self->client_, handle});
99
100 try {
101     self->client_->async_get_result(handle, result_handler{self});
102 } catch (...) {
103     self->stop_callback_->reset();
104     stdexec::set_error(std::move(self->receiver_), std::current_exception());
105 }
106 }
107 };
108
109 template <typename T>
110 template <typename R>
111 struct execute_action_sender<T>::opstate<R>::result_handler {
112     opstate* self;

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

57     }
58 }
59
60 private:
61     struct response_handler;
62     struct result_handler;
63
64     struct stop_handler {
65         std::shared_ptr<client_type> client;
66         std::shared_ptr<goal_handle_type> handle;
67
68         void operator()() noexcept { client->async_cancel_goal(handle); }
69     };
70
71     using stop_token = stdexec::stop_token_of_t<stdexec::env_of_t<R>>;
72     using stop_callback = stdexec::stop_callback_for_t<stop_token, stop_handler>;
73     using stop_callback_storage = std::conditional_t<
74         stdexec::unstoppable_token<stop_token>,
75         empty_optional,
76         std::optional<stop_callback>;

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

101     self->client_->async_get_result(handle, result_handler{self});
102 } catch (...) {
103     self->stop_callback_->reset();
104     stdexec::set_error(std::move(self->receiver_), std::current_exception());
105 }
106 }
107 };
108
109 template <typename T>
110 template <typename R>
111 struct execute_action_sender<T>::opstate<R>::result_handler {
112     opstate* self;
113
114     void operator()(const typename goal_handle_type::WrappedResult& wrapped_result) noexcept {
115         self->stop_callback_.reset();
116
117         switch (wrapped_result.code) {
118             case rclcpp_action::ResultCode::SUCCEEDED:
119                 stdexec::set_value(std::move(self->receiver_), wrapped_result.result);

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

103     self->stop_callback_->reset();
104     stdexec::set_error(std::move(self->receiver_), std::current_exception());
105 }
106 }
107 };
108
109 template <typename T>
110 template <typename R>
111 struct execute_action_sender<T>::opstate<R>::result_handler {
112     opstate* self;
113
114     void operator()(const typename goal_handle_type::WrappedResult& wrapped_result) noexcept {
115         self->stop_callback_.reset();
116
117         switch (wrapped_result.code) {
118             case rclcpp_action::ResultCode::SUCCEEDED:
119                 stdexec::set_value(std::move(self->receiver_), wrapped_result.result);
120                 break;
121             case rclcpp_action::ResultCode::CANCELED:

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```
105     }
106   }
107 };
108
109 template <typename T>
110 template <typename R>
111 struct execute_action_sender<T>::opstate<R>::result_handler {
112   opstate* self;
113
114   void operator()(const typename goal_handle_type::WrappedResult& wrapped_result) noexcept {
115     self->stop_callback_.reset();
116
117     switch (wrapped_result.code) {
118       case rclcpp_action::ResultCode::SUCCEEDED:
119         stdexec::set_value(std::move(self->receiver_), wrapped_result.result);
120         break;
121       case rclcpp_action::ResultCode::CANCELED:
122         stdexec::set_stopped(std::move(self->receiver_));
123         break;
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

106     }
107 };
108
109 template <typename T>
110 template <typename R>
111 struct execute_action_sender<T>::opstate<R>::result_handler {
112     opstate* self;
113
114     void operator()(const typename goal_handle_type::WrappedResult& wrapped_result) noexcept {
115         self->stop_callback_.reset();
116
117         switch (wrapped_result.code) {
118             case rclcpp_action::ResultCode::SUCCEEDED:
119                 stdexec::set_value(std::move(self->receiver_), wrapped_result.result);
120                 break;
121             case rclcpp_action::ResultCode::CANCELED:
122                 stdexec::set_stopped(std::move(self->receiver_));
123                 break;
124             case rclcpp_action::ResultCode::ABORTED:

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```
108
109 template <typename T>
110 template <typename R>
111 struct execute_action_sender<T>::opstate<R>::result_handler {
112     opstate* self;
113
114     void operator()(const typename goal_handle_type::WrappedResult& wrapped_result) noexcept {
115         self->stop_callback_.reset();
116
117         switch (wrapped_result.code) {
118             case rclcpp_action::ResultCode::SUCCEEDED:
119                 stdexec::set_value(std::move(self->receiver_), wrapped_result.result);
120                 break;
121             case rclcpp_action::ResultCode::CANCELED:
122                 stdexec::set_stopped(std::move(self->receiver_));
123                 break;
124             case rclcpp_action::ResultCode::ABORTED:
125                 stdexec::set_error(
126                     std::move(self->receiver_),
```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

109 template <typename T>
110 template <typename R>
111 struct execute_action_sender<T>::opstate<R>::result_handler {
112     opstate* self;
113
114     void operator()(const typename goal_handle_type::WrappedResult& wrapped_result) noexcept {
115         self->stop_callback_.reset();
116
117         switch (wrapped_result.code) {
118             case rclcpp_action::ResultCode::SUCCEEDED:
119                 stdexec::set_value(std::move(self->receiver_), wrapped_result.result);
120                 break;
121             case rclcpp_action::ResultCode::CANCELED:
122                 stdexec::set_stopped(std::move(self->receiver_));
123                 break;
124             case rclcpp_action::ResultCode::ABORTED:
125                 stdexec::set_error(
126                     std::move(self->receiver_),
127                     std::make_exception_ptr(action_aborted("goal aborted by the server")));
128                 break;

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

112 opstate = set1,
113
114 void operator()(const typename goal_handle_type::WrappedResult& wrapped_result) noexcept {
115     self->stop_callback_.reset();
116
117     switch (wrapped_result.code) {
118         case rclcpp_action::ResultCode::SUCCEEDED:
119             stdexec::set_value(std::move(self->receiver_), wrapped_result.result);
120             break;
121         case rclcpp_action::ResultCode::CANCELED:
122             stdexec::set_stopped(std::move(self->receiver_));
123             break;
124         case rclcpp_action::ResultCode::ABORTED:
125             stdexec::set_error(
126                 std::move(self->receiver_),
127                 std::make_exception_ptr(action_aborted("goal aborted by the server")));
128             break;
129         default:
130             stdexec::set_error(
131                 std::move(self->receiver_),

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

110
117 switch (wrapped_result.code) {
118     case rclcpp_action::ResultCode::SUCCEEDED:
119         stdexec::set_value(std::move(self->receiver_), wrapped_result.result);
120         break;
121     case rclcpp_action::ResultCode::CANCELED:
122         stdexec::set_stopped(std::move(self->receiver_));
123         break;
124     case rclcpp_action::ResultCode::ABORTED:
125         stdexec::set_error(
126             std::move(self->receiver_),
127             std::make_exception_ptr(action_aborted("goal aborted by the server")));
128         break;
129     default:
130         stdexec::set_error(
131             std::move(self->receiver_),
132             std::make_exception_ptr(action_unknown_result_code("unknown error code")));
133         break;
134 }
135

```

sender

opstate

response
handler

stop
handler

result
handler

ADAPTER LAYER

rclexec::execute_action_sender

```

121     case rclcpp_action::ResultCode::CANCELLED:
122         stdexec::set_stopped(std::move(self->receiver_));
123         break;
124     case rclcpp_action::ResultCode::ABORTED:
125         stdexec::set_error(
126             std::move(self->receiver_),
127             std::make_exception_ptr(action_aborted("goal aborted by the server")));
128         break;
129     default:
130         stdexec::set_error(
131             std::move(self->receiver_),
132             std::make_exception_ptr(action_unknown_result_code("unknown error code")));
133         break;
134     }
135 }
136 };
137
138 template <typename T>
139 template <typename R>
140 auto execute_action_sender(T::connect(R receiver), const noexcept > opstate<R> f

```

sender

opstate

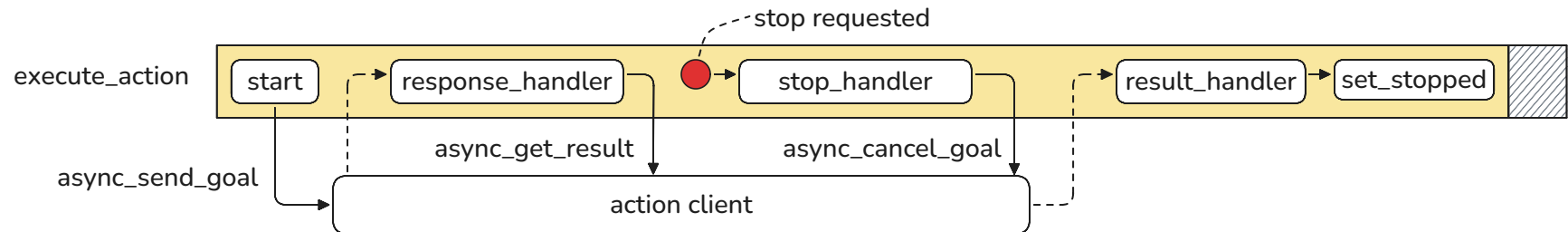
response
handler

stop
handler

result
handler

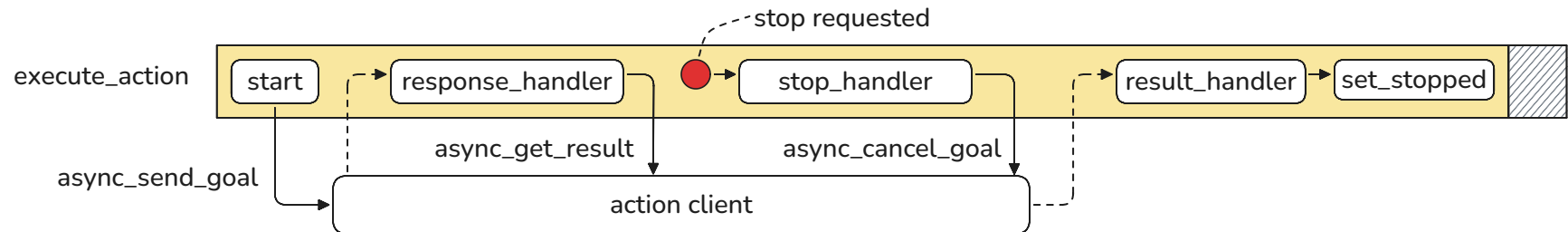
ADAPTER LAYER

`rclexec::execute_action`



ADAPTER LAYER

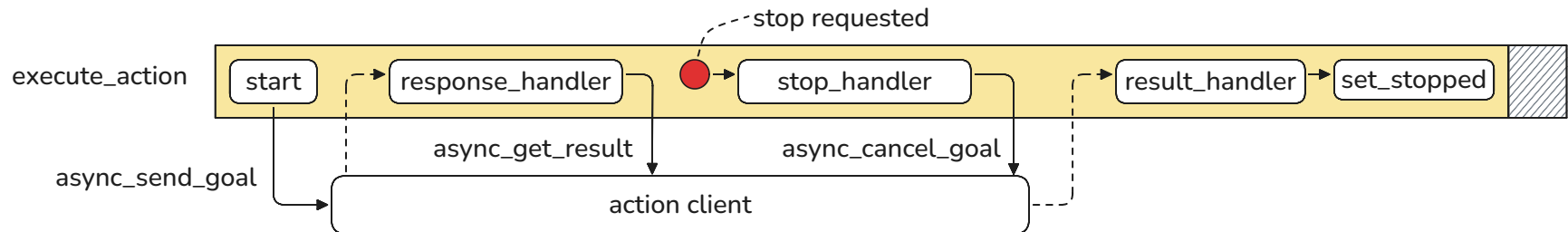
`rclexec::execute_action`



- Server resolves the race between cancellation and completion.

ADAPTER LAYER

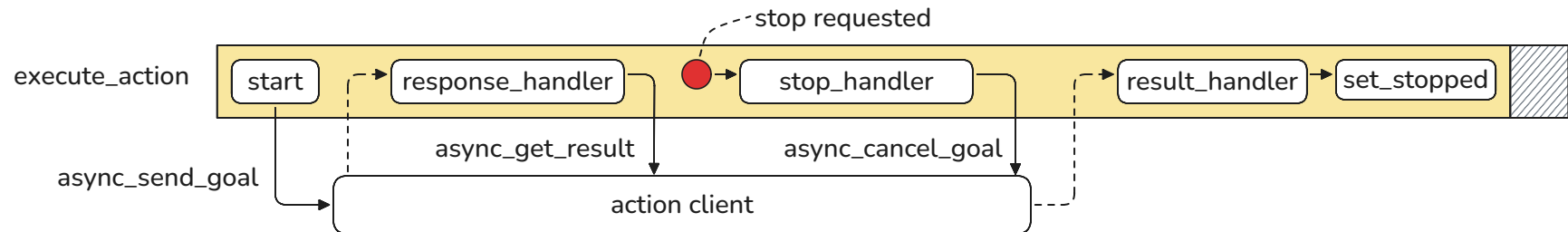
`rclexec::execute_action`



- Server resolves the race between cancellation and completion.
- Before acceptance there is no remote goal to cancel.

ADAPTER LAYER

`rclexec::execute_action`



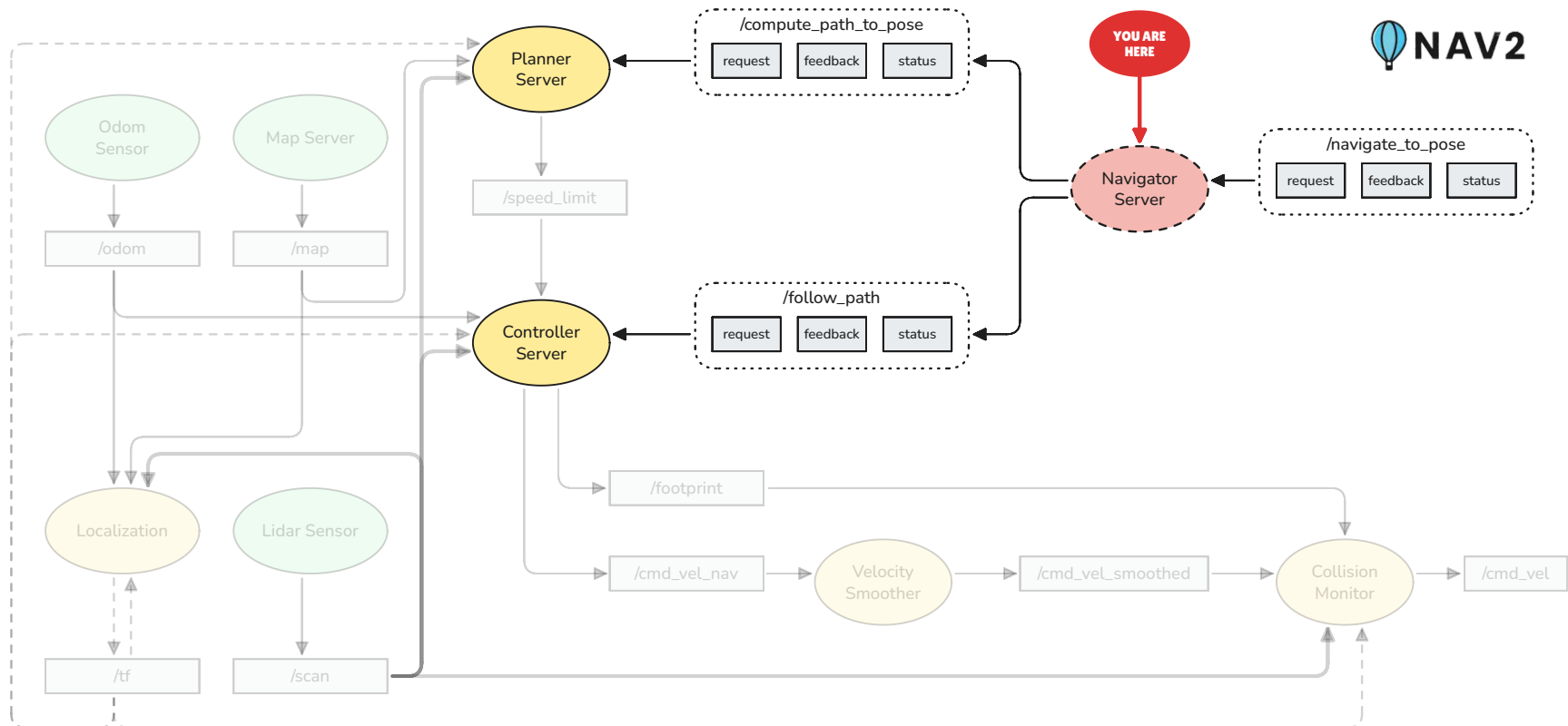
- Server resolves the race between cancellation and completion.
- Before acceptance there is no remote goal to cancel.
- Missing action feedback support.

ADAPTER LAYER

rclexec::timer_scheduler

```
1 auto sched = rclexec::timer_scheduler(node);  
2  
3 auto work = exec::schedule_after(sched, 1s)  
4     | stdexec::let_value([] {  
5         return delayed_operation_sender();  
6     });
```

CASE STUDY



CASE STUDY

```
1 int main(int argc, char** argv) {
2     rclcpp::init(argc, argv);
3
4     auto node = std::make_shared<rclcpp::Node>("navigator");
5     auto navigator = example::navigator{node};
6
7     auto server = rclcpp::make_action_server_preempt<example::NavigateToPose>(
8         node,
9         "navigate_to_pose",
10        [&navigator](auto mission) { return navigator(mission); });
11
12    rclcpp::spin(node);
13    rclcpp::shutdown();
14    return 0;
15 }
```

CASE STUDY

```
1 int main(int argc, char** argv) {
2     rclcpp::init(argc, argv);
3
4     auto node = std::make_shared<rclcpp::Node>("navigator");
5     auto navigator = example::navigator{node};
6
7     auto server = rclcpp::make_action_server_preempt<example::NavigateToPose>(
8         node,
9         "navigate_to_pose",
10        [&navigator](auto mission) { return navigator(mission); });
11
12    rclcpp::spin(node);
13    rclcpp::shutdown();
14    return 0;
15 }
```

CASE STUDY

```
1 int main(int argc, char** argv) {
2     rclcpp::init(argc, argv);
3
4     auto node = std::make_shared<rclcpp::Node>("navigator");
5     auto navigator = example::navigator{node};
6
7     auto server = rclcpp::make_action_server_preempt<example::NavigateToPose>(
8         node,
9         "navigate_to_pose",
10        [&navigator](auto mission) { return navigator(mission); });
11
12    rclcpp::spin(node);
13    rclcpp::shutdown();
14    return 0;
15 }
```

- Server takes a sender factory to call when a new goal is accepted.

CASE STUDY

```
1 int main(int argc, char** argv) {
2     rclcpp::init(argc, argv);
3
4     auto node = std::make_shared<rclcpp::Node>("navigator");
5     auto navigator = example::navigator{node};
6
7     auto server = rclcpp::make_action_server_preempt<example::NavigateToPose>(
8         node,
9         "navigate_to_pose",
10        [&navigator](auto mission) { return navigator(mission); });
11
12    rclcpp::spin(node);
13    rclcpp::shutdown();
14    return 0;
15 }
```

- Server takes a sender factory to call when a new goal is accepted.
- Clients can cancel the goal, and the server can preempt it.

CASE STUDY

```
1 int main(int argc, char** argv) {  
2     rclcpp::init(argc, argv);  
3  
4     auto node = std::make_shared<rclcpp::Node>("navigator");  
5     auto navigator = example::navigator{node};  
6  
7     auto server = rclexec::make_action_server_preempt<example::NavigateToPose>(  
8         node,  
9         "navigate_to_pose",  
10        [&navigator](auto mission) { return navigator(mission); });  
11  
12    rclcpp::spin(node);  
13    rclcpp::shutdown();  
14    return 0;  
15 }
```

- Server takes a sender factory to call when a new goal is accepted.
- Clients can cancel the goal, and the server can preempt it.
- Completion signals are mapped to the action response API.

set_value(receiver, result)	→	handle->succeed(result)
-----------------------------	---	-------------------------

set_error(receiver, exception)	→	handle->abort()
--------------------------------	---	-----------------

set_stopped(receiver)	→	handle->canceled() for client cancellation; handle->abort() for server stop/preemption
-----------------------	---	---

CASE STUDY

```
1 class navigator {
2   public:
3     using goal_handle_type = rclcpp_action::ServerGoalHandle<NavigateToPose>;
4     using result_type = NavigateToPose::Result;
5
6     navigator(std::shared_ptr<rclcpp::Node> node)
7       : sched_(node),
8         planner_(rclcpp_action::create_client<ComputePathToPose>(node, "compute_path_to_pose"),
9         controller_(rclcpp_action::create_client<FollowPath>(node, "follow_path")),
10        spin_(rclcpp_action::create_client<Spin>(node, "spin")) {}
11
12     auto operator()(std::shared_ptr<goal_handle_type> mission) const
13       -> std::future<std::shared_ptr<result_type>>();
14
15   private:
16     rclcpp::timer_schduler sched_;
17     std::shared_ptr<rclcpp_action::Client<ComputePathToPose>> planner_;
18     std::shared_ptr<rclcpp_action::Client<FollowPath>> controller_;
19     std::shared_ptr<rclcpp_action::Client<Spin>> spin_;
```

CASE STUDY

```
1 class navigator {
2   public:
3     using goal_handle_type = rclcpp_action::ServerGoalHandle<NavigateToPose>;
4     using result_type = NavigateToPose::Result;
5
6     navigator(std::shared_ptr<rclcpp::Node> node)
7       : sched_(node),
8         planner_(rclcpp_action::create_client<ComputePathToPose>(node, "compute_path_to_pose"),
9         controller_(rclcpp_action::create_client<FollowPath>(node, "follow_path")),
10        spin_(rclcpp_action::create_client<Spin>(node, "spin")) {}
11
12     auto operator()(std::shared_ptr<goal_handle_type> mission) const
13       -> std::exec::task<std::shared_ptr<result_type>>;
14
15   private:
16     rclexec::timer_scheduler sched_;
17     std::shared_ptr<rclcpp_action::Client<ComputePathToPose>> planner_;
18     std::shared_ptr<rclcpp_action::Client<FollowPath>> controller_;
19     std::shared_ptr<rclcpp_action::Client<Spin>> spin_;
```

CASE STUDY

```
1 class navigator {
2   public:
3     using goal_handle_type = rclcpp_action::ServerGoalHandle<NavigateToPose>;
4     using result_type = NavigateToPose::Result;
5
6     navigator(std::shared_ptr<rclcpp::Node> node)
7       : sched_(node),
8         planner_(rclcpp_action::create_client<ComputePathToPose>(node, "compute_path_to_pose"),
9         controller_(rclcpp_action::create_client<FollowPath>(node, "follow_path")),
10        spin_(rclcpp_action::create_client<Spin>(node, "spin")) {}
11
12     auto operator()(std::shared_ptr<goal_handle_type> mission) const
13       -> std::future<std::shared_ptr<result_type>>();
14
15   private:
16     rclcpp::timer_schduler sched_;
17     std::shared_ptr<rclcpp_action::Client<ComputePathToPose>> planner_;
18     std::shared_ptr<rclcpp_action::Client<FollowPath>> controller_;
19     std::shared_ptr<rclcpp_action::Client<Spin>> spin_;
```

CASE STUDY

```
2 public:
3     using goal_handle_type = rclcpp_action::ServerGoalHandle<NavigateToPose>;
4     using result_type = NavigateToPose::Result;
5
6     navigator(std::shared_ptr<rclcpp::Node> node)
7         : sched_(node),
8           planner_(rclcpp_action::create_client<ComputePathToPose>(node, "compute_path_to_pose"),
9                   controller_(rclcpp_action::create_client<FollowPath>(node, "follow_path")),
10                   spin_(rclcpp_action::create_client<Spin>(node, "spin"))) {}
11
12     auto operator()(std::shared_ptr<goal_handle_type> mission) const
13         -> std::exec::task<std::shared_ptr<result_type>>;
14
15 private:
16     rclexec::timer_scheduler sched_;
17     std::shared_ptr<rclcpp_action::Client<ComputePathToPose>> planner_;
18     std::shared_ptr<rclcpp_action::Client<FollowPath>> controller_;
19     std::shared_ptr<rclcpp_action::Client<Spin>> spin_;
20 };
```

CASE STUDY

```
1 class navigator {
2   public:
3     using goal_handle_type = rclcpp_action::ServerGoalHandle<NavigateToPose>;
4     using result_type = NavigateToPose::Result;
5
6     navigator(std::shared_ptr<rclcpp::Node> node)
7       : sched_(node),
8         planner_(rclcpp_action::create_client<ComputePathToPose>(node, "compute_path_to_pose"),
9         controller_(rclcpp_action::create_client<FollowPath>(node, "follow_path")),
10        spin_(rclcpp_action::create_client<Spin>(node, "spin")) {}
11
12     auto operator()(std::shared_ptr<goal_handle_type> mission) const
13       -> stdexec::task<std::shared_ptr<result_type>>;
14
15   private:
16     rclexec::timer_scheduler sched_;
17     std::shared_ptr<rclcpp_action::Client<ComputePathToPose>> planner_;
18     std::shared_ptr<rclcpp_action::Client<FollowPath>> controller_;
19     std::shared_ptr<rclcpp_action::Client<Spin>> spin_;
```

- `stdexec::task` is both a sender and a valid coroutine return type.

CASE STUDY

1. Call the planning action.
2. Pass the result to the controller action.
3. If we didn't reach the goal in 1 second, replan!

```
1 auto navigator::operator()(std::shared_ptr<goal_handle_type> mission) const -> std::exec::task<
2   ComputePathToPose::Goal planner_goal;
3   planner_goal.goal = mission->get_goal()->pose;
4   planner_goal.use_start = false;
5   planner_goal.planner_id = kPlannerId;
6
7   auto plan = co_await rclexec::execute_action(this->planner_, planner_goal);
8
9   while (plan) {
10    FollowPath::Goal controller_goal;
11    controller_goal.path = std::move(plan->path);
12    controller_goal.controller_id = kControllerId;
13
14    auto follow_task =
15      rclexec::execute_action(this->controller_, std::move(controller_goal))
16      | std::exec::then([](auto&&) -> decltype(plan) { return nullptr; });
17
18    auto replan_task =
19      exec::schedule_after(this->sched, 1s)
```

CASE STUDY

1. Call the planning action.
2. Pass the result to the controller action.
3. If we didn't reach the goal in 1 second, replan!

```
1 auto navigator::operator()(std::shared_ptr<goal_handle_type> mission) const -> std::exec::task<
2   ComputePathToPose::Goal planner_goal;
3   planner_goal.goal = mission->get_goal()->pose;
4   planner_goal.use_start = false;
5   planner_goal.planner_id = kPlannerId;
6
7   auto plan = co_await rclexec::execute_action(this->planner_, planner_goal);
8
9   while (plan) {
10     FollowPath::Goal controller_goal;
11     controller_goal.path = std::move(plan->path);
12     controller_goal.controller_id = kControllerId;
13
14     auto follow_task =
15       rclexec::execute_action(this->controller_, std::move(controller_goal))
16       | std::exec::then([](auto&&) -> decltype(plan) { return nullptr; });
17
18     auto replan_task =
19       exec::schedule_after(this->sched, 1s)
```

CASE STUDY

1. Call the planning action.
2. Pass the result to the controller action.
3. If we didn't reach the goal in 1 second, replan!

```
1 auto navigator::operator()(std::shared_ptr<goal_handle_type> mission) const -> std::exec::task<
2   ComputePathToPose::Goal planner_goal;
3   planner_goal.goal = mission->get_goal()->pose;
4   planner_goal.use_start = false;
5   planner_goal.planner_id = kPlannerId;
6
7   auto plan = co_await rclexec::execute_action(this->planner_, planner_goal);
8
9   while (plan) {
10    FollowPath::Goal controller_goal;
11    controller_goal.path = std::move(plan->path);
12    controller_goal.controller_id = kControllerId;
13
14    auto follow_task =
15      rclexec::execute_action(this->controller_, std::move(controller_goal))
16      | std::exec::then([](auto&&) -> decltype(plan) { return nullptr; });
17
18    auto replan_task =
19      exec::schedule_after(this->sched, 1s)
```

CASE STUDY

1. Call the planning action.
2. Pass the result to the controller action.
3. If we didn't reach the goal in 1 second, replan!

```

1  auto navigator::operator()(std::shared_ptr<goal_handle_type> mission) const -> stdexec::task<
2    ComputePathToPose::Goal planner_goal;
3    planner_goal.goal = mission->get_goal()->pose;
4    planner_goal.use_start = false;
5    planner_goal.planner_id = kPlannerId;
6
7    auto plan = co_await rclexec::execute_action(this->planner_, planner_goal);
8
9    while (plan) {
10       FollowPath::Goal controller_goal;
11       controller_goal.path = std::move(plan->path);
12       controller_goal.controller_id = kControllerId;
13
14       auto follow_task =
15         rclexec::execute_action(this->controller_, std::move(controller_goal))
16         | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });
17
18       auto replan_task =
19         exec::schedule_after(this->sched, 1s)

```

CASE STUDY

1. Call the planning action.
2. Pass the result to the controller action.
3. If we didn't reach the goal in 1 second, replan!

```
2  ComputePathToPose::Goal planner_goal;  
3  planner_goal.goal = mission->get_goal()->pose;  
4  planner_goal.use_start = false;  
5  planner_goal.planner_id = kPlannerId;  
6  
7  auto plan = co_await rclexec::execute_action(this->planner_, planner_goal);  
8  
9  while (plan) {  
10     FollowPath::Goal controller_goal;  
11     controller_goal.path = std::move(plan->path);  
12     controller_goal.controller_id = kControllerId;  
13  
14     auto follow_task =  
15         rclexec::execute_action(this->controller_, std::move(controller_goal))  
16         | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });  
17  
18     auto replan_task =  
19         exec::schedule_after(this->sched_, 1s)  
20         | stdexec::let_value([this, planner_goal] {
```

CASE STUDY

1. Call the planning action.
2. Pass the result to the controller action.
3. If we didn't reach the goal in 1 second, replan!

```

10 FollowPath::Goal controller_goal;
11 controller_goal.path = std::move(plan->path);
12 controller_goal.controller_id = kControllerId;
13
14 auto follow_task =
15     rclexec::execute_action(this->controller_, std::move(controller_goal))
16     | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });
17
18 auto replan_task =
19     exec::schedule_after(this->sched_, 1s)
20     | stdexec::let_value([this, planner_goal] {
21         return rclexec::execute_action(this->planner_, planner_goal);
22     });
23
24 plan = co_await exec::when_any(std::move(follow_task), std::move(replan_task));
25 }
26
27 co_return std::make_shared<result_type>();
28 }

```

CASE STUDY

1. Call the planning action.
2. Pass the result to the controller action.
3. If we didn't reach the goal in 1 second, replan!

```
6
7  auto plan = co_await rclexec::execute_action(this->planner_, planner_goal);
8
9  while (plan) {
10     FollowPath::Goal controller_goal;
11     controller_goal.path = std::move(plan->path);
12     controller_goal.controller_id = kControllerId;
13
14     auto follow_task =
15         rclexec::execute_action(this->controller_, std::move(controller_goal))
16         | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });
17
18     auto replan_task =
19         exec::schedule_after(this->sched_, 1s)
20         | stdexec::let_value([this, planner_goal] {
21             return rclexec::execute_action(this->planner_, planner_goal);
22         });
23
24     plan = co_await exec::when_any(std::move(follow_task), std::move(replan_task));
```

CASE STUDY

1. Call the planning action.
2. Pass the result to the controller action.
3. If we didn't reach the goal in 1 second, replan!

```

10 FollowPath::Goal controller_goal;
11 controller_goal.path = std::move(plan->path);
12 controller_goal.controller_id = kControllerId;
13
14 auto follow_task =
15     rclexec::execute_action(this->controller_, std::move(controller_goal))
16     | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });
17
18 auto replan_task =
19     exec::schedule_after(this->sched_, 1s)
20     | stdexec::let_value([this, planner_goal] {
21         return rclexec::execute_action(this->planner_, planner_goal);
22     });
23
24 plan = co_await exec::when_any(std::move(follow_task), std::move(replan_task));
25 }
26
27 co_return std::make_shared<result_type>();
28 }

```

CASE STUDY

1. Call the planning action.
2. Pass the result to the controller action.
3. If we didn't reach the goal in 1 second, replan!

```

10 FollowPath::Goal controller_goal;
11 controller_goal.path = std::move(plan->path);
12 controller_goal.controller_id = kControllerId;
13
14 auto follow_task =
15     rclexec::execute_action(this->controller_, std::move(controller_goal))
16     | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });
17
18 auto replan_task =
19     exec::schedule_after(this->sched_, 1s)
20     | stdexec::let_value([this, planner_goal] {
21         return rclexec::execute_action(this->planner_, planner_goal);
22     });
23
24 plan = co_await exec::when_any(std::move(follow_task), std::move(replan_task));
25 }
26
27 co_return std::make_shared<result_type>();
28 }

```

CASE STUDY

ERROR HANDLING

```
1 ComputePathToPose::Result::NONE
2 ComputePathToPose::Result::GOAL_REJECTED
3 ComputePathToPose::Result::SEND_GOAL_FAILURE
4 ComputePathToPose::Result::UNKNOWN
5 ComputePathToPose::Result::INVALID_PLANNER
6 ComputePathToPose::Result::TF_ERROR
7 ComputePathToPose::Result::START_OUTSIDE_MAP
8 ComputePathToPose::Result::GOAL_OUTSIDE_MAP
9 ComputePathToPose::Result::START_OCCUPIED
10 ComputePathToPose::Result::GOAL_OCCUPIED
11 ComputePathToPose::Result::TIMEOUT
12 ComputePathToPose::Result::NO_VALID_PATH
```

```
1 FollowPath::Result::NONE
2 FollowPath::Result::GOAL_REJECTED
3 FollowPath::Result::SEND_GOAL_FAILURE
4 FollowPath::Result::UNKNOWN
5 FollowPath::Result::INVALID_CONTROLLER
6 FollowPath::Result::TF_ERROR
7 FollowPath::Result::INVALID_PATH
8 FollowPath::Result::PATIENCE_EXCEEDED
9 FollowPath::Result::FAILED_TO_MAKE_PROGRESS
10 FollowPath::Result::NO_VALID_CONTROL
11 FollowPath::Result::CONTROLLER_TIMED_OUT
12 FollowPath::Result::TIMEOUT
```

CASE STUDY

ERROR HANDLING

```
1 template <typename T>
2 auto forward_unless_error(std::shared_ptr<T> result) {
3     if (result->error_code != 0) {
4         throw navigation_failed{result->error_msg, result->error_code};
5     }
6     return result;
7 }
8
9 auto navigator::operator()(std::shared_ptr<goal_handle_type> mission) const -> stdexec::task<
10     try {
11         ComputePathToPose::Goal planner_goal;
12         planner_goal.goal = mission->get_goal()->pose;
13         planner_goal.use_start = false;
14         planner_goal.planner_id = kPlannerId;
15
16         auto plan = co_await (rclexec::execute_action(this->planner_, planner_goal)
17             | stdexec::then(forward_unless_error<ComputePathToPose::Result>));
18
19         while (plan) {
```

CASE STUDY

ERROR HANDLING

```
1 template <typename T>
2 auto forward_unless_error(std::shared_ptr<T> result) {
3     if (result->error_code != 0) {
4         throw navigation_failed{result->error_msg, result->error_code};
5     }
6     return result;
7 }
8
9 auto navigator::operator()(std::shared_ptr<goal_handle_type> mission) const -> stdexec::task<
10 try {
11     ComputePathToPose::Goal planner_goal;
12     planner_goal.goal = mission->get_goal()->pose;
13     planner_goal.use_start = false;
14     planner_goal.planner_id = kPlannerId;
15
16     auto plan = co_await (rclexec::execute_action(this->planner_, planner_goal)
17         | stdexec::then(forward_unless_error<ComputePathToPose::Result>));
18
19     while (plan) {
```

CASE STUDY

ERROR HANDLING

```
21 controller_goal.path = std::move(plan->path);
22 controller_goal.controller_id = kControllerId;
23
24 auto follow_task =
25     rclexec::execute_action(this->controller_, std::move(controller_goal))
26     | stdexec::then(forward_unless_error<FollowPath::Result>)
27     | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });
28
29 auto replan_task =
30     exec::schedule_after(this->sched_, 1s)
31     | stdexec::let_value([this, planner_goal] {
32         return rclexec::execute_action(this->planner_, planner_goal);
33     })
34     | stdexec::then(forward_unless_error<ComputePathToPose::Result>);
35
36 plan = co_await exec::when_any(std::move(follow_task), std::move(replan_task));
37 }
38
39 co_return std::make_shared<result_type>();
```

CASE STUDY

ERROR HANDLING

```
32         return rclexec::execute_action(this->planner_, planner_goal);
33     })
34     | stdexec::then(forward_unless_error<ComputePathToPose::Result>);
35
36     plan = co_await exec::when_any(std::move(follow_task), std::move(replan_task));
37 }
38
39     co_return std::make_shared<result_type>();
40 } catch (const example::navigation_failed&) {
41 } catch (const rclexec::action_aborted&) {
42 }
43
44 Spin::Goal spin_goal;
45 spin_goal.target_yaw = kSpinDistance;
46 spin_goal.time_allowance = rclcpp::Duration(kBehaviorAllowance);
47
48 while (true) {
49     RCLCPP_ERROR(rclcpp::get_logger("rclexec"), "HELP!");
50     co_await rclexec::execute_action(this->spin_, spin_goal);
```

CASE STUDY

ERROR HANDLING

```

34         | std::exec::then(forward_unless_error<ComputePathToPose::Result>);
35
36     plan = co_await exec::when_any(std::move(follow_task), std::move(replan_task));
37 }
38
39     co_return std::make_shared<result_type>();
40 } catch (const example::navigation_failed&) {
41 } catch (const rclexec::action_aborted&) {
42 }
43
44 Spin::Goal spin_goal;
45 spin_goal.target_yaw = kSpinDistance;
46 spin_goal.time_allowance = rclcpp::Duration(kBehaviorAllowance);
47
48 while (true) {
49     RCLCPP_ERROR(rclcpp::get_logger("rclexec"), "HELP!");
50     co_await rclexec::execute_action(this->spin_, spin_goal);
51 }
52 }

```

CASE STUDY

ERROR HANDLING

```

34         | std::exec::then(forward_unless_error<ComputePathToPose::Result>);
35
36     plan = co_await exec::when_any(std::move(follow_task), std::move(replan_task));
37 }
38
39     co_return std::make_shared<result_type>();
40 } catch (const example::navigation_failed&) {
41 } catch (const rclexec::action_aborted&) {
42 }
43
44 Spin::Goal spin_goal;
45 spin_goal.target_yaw = kSpinDistance;
46 spin_goal.time_allowance = rclcpp::Duration(kBehaviorAllowance);
47
48 while (true) {
49     RCLCPP_ERROR(rclcpp::get_logger("rclexec"), "HELP!");
50     co_await rclexec::execute_action(this->spin_, spin_goal);
51 }
52 }

```

CASE STUDY

```
1 template <typename T>
2 auto forward_unless_error(std::shared_ptr<T> result) {
3     if (result->error_code != 0) {
4         throw navigation_failed{result->error_msg, result->error_code};
5     }
6     return result;
7 }
8
9 auto navigator::operator()(std::shared_ptr<goal_handle_type> mission) const -> stdexec::task<std::shared_ptr<result_type>> {
10     try {
11         ComputePathToPose::Goal planner_goal;
12         planner_goal.goal = mission->get_goal()->pose;
13         planner_goal.use_start = false;
14         planner_goal.planner_id = kPlannerId;
15
16         auto plan = co_await (rclexec::execute_action(this->planner_, planner_goal)
17             | stdexec::then(forward_unless_error<ComputePathToPose::Result>));
18
19         while (plan) {
20             FollowPath::Goal controller_goal;
21             controller_goal.path = std::move(plan->path);
22             controller_goal.controller_id = kControllerId;
23
24             auto follow_task =
25                 rclexec::execute_action(this->controller_, std::move(controller_goal))
26                 | stdexec::then(forward_unless_error<FollowPath::Result>)
27                 | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });
28
29             auto replan_task =
30                 exec::schedule_after(this->sched_, 1s)
31                 | stdexec::let_value([this, planner_goal] {
32                     return rclexec::execute_action(this->planner_, planner_goal);
33                 })
34                 | stdexec::then(forward_unless_error<ComputePathToPose::Result>);
35
36             plan = co_await exec::when_any(std::move(follow_task), std::move(replan_task));
37         }
38
39         co_return std::make_shared<result_type>();
40     } catch (const example::navigation_failed&) {
41     } catch (const rclexec::action_aborted&) {
42     }
43
44     Spin::Goal spin_goal;
45     spin_goal.target_yaw = kSpinDistance;
46     spin_goal.time_allowance = rclcpp::Duration(kBehaviorAllowance);
47 }
```

CASE STUDY

SUMMARY

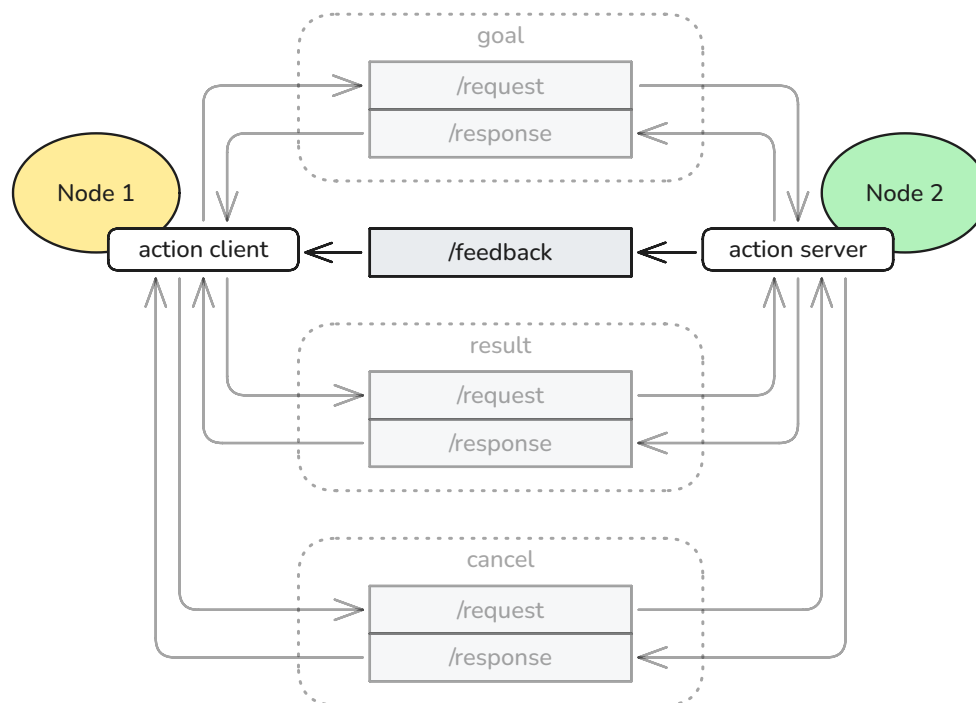
~50 lines of code to implement a navigation task that:

- Handles automatic replanning
- Supports cooperative cancellation
- Includes “graceful” recovery
- Uses purely generic async algorithms

Exceptions and coroutines are opt-in.

BONUS

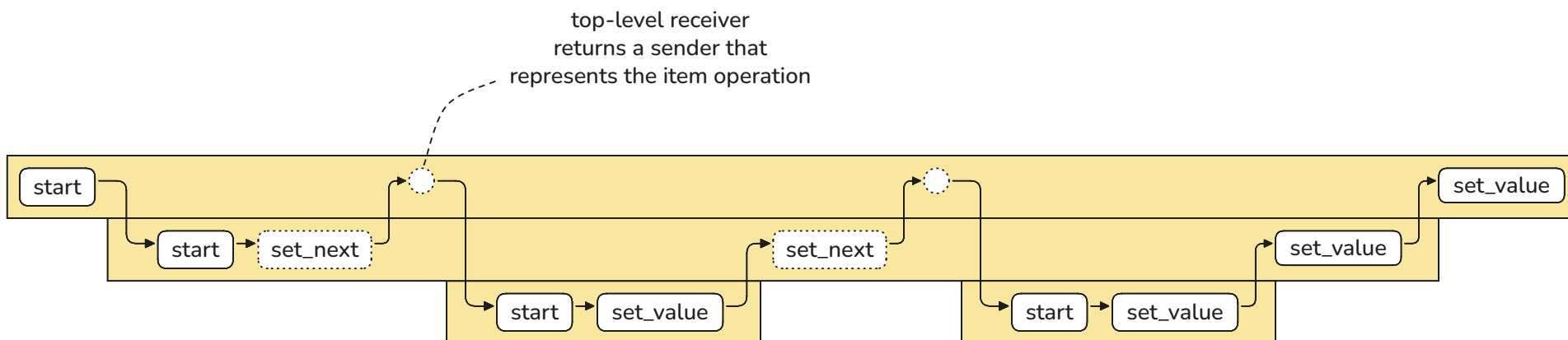
ROS2



BONUS

SEQUENCE SENDERS

Multi-shot senders call `set_next`



BONUS

SEQUENCE SENDERS

`rclexec::experimental::execute_action_with_feedback`

```
1 template <typename T>
2 auto forward_unless_error(std::shared_ptr<T> result) {
3     if (result->error_code != 0) {
4         throw navigation_failed{result->error_msg, result->error_code};
5     }
6     return result;
7 }
8
9 auto navigator::operator()(std::shared_ptr<goal_handle_type> mission) const -> stdexec::task<
10     try {
11         auto started = this->clock_->now();
12
13         ComputePathToPose::Goal planner_goal;
14         planner_goal.goal = mission->get_goal()->pose;
15         planner_goal.use_start = false;
16         planner_goal.planner_id = kPlannerId;
17
18         auto plan = co_await (rclexec::execute_action(this->planner_, planner_goal)
19             | stdexec::then(forward_unless_error<ComputePathToPose::Result>));
```

BONUS

SEQUENCE SENDERS

`rclexec::experimental::execute_action_with_feedback`

```

19 | stdexec::then(forward_unless_error<ComputePathToPose::Result>));
20
21 while (plan) {
22   FollowPath::Goal controller_goal;
23   controller_goal.path = std::move(plan->path);
24   controller_goal.controller_id = kControllerId;
25
26   auto follow_task =
27     rclexec::experimental::execute_action_with_feedback(this->controller_, std::move(cont
28     | rclexec::experimental::transform_each(
29       stdexec::then([this, mission, started](auto item) noexcept {
30         NavigateToPose::Feedback message;
31         message.navigation_time = this->clock_->now() - started;
32         message.distance_remaining = item->distance_to_goal;
33         mission->publish_feedback(std::make_shared<NavigateToPose::Feedback>(message)
34       })))
35   | stdexec::then(forward_unless_error<FollowPath::Result>)
36   | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });
37

```

BONUS

SEQUENCE SENDERS

rclexec::experimental::execute_action_with_feedback

```
20
21 while (plan) {
22     FollowPath::Goal controller_goal;
23     controller_goal.path = std::move(plan->path);
24     controller_goal.controller_id = kControllerId;
25
26     auto follow_task =
27         rclexec::experimental::execute_action_with_feedback(this->controller_, std::move(controller_goal),
28             | rclexec::experimental::transform_each(
29                 stdexec::then([this, mission, started](auto item) noexcept {
30                     NavigateToPose::Feedback message;
31                     message.navigation_time = this->clock_->now() - started;
32                     message.distance_remaining = item->distance_to_goal;
33                     mission->publish_feedback(std::make_shared<NavigateToPose::Feedback>(message))
34                 }))
35         | stdexec::then(forward_unless_error<FollowPath::Result>)
36         | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });
37
38     auto replan_task =
```

BONUS

SEQUENCE SENDERS

`rclexec::experimental::execute_action_with_feedback`

```

22 FollowPath::Result controller_goal,
23 controller_goal.path = std::move(plan->path);
24 controller_goal.controller_id = kControllerId;
25
26 auto follow_task =
27 rclexec::experimental::execute_action_with_feedback(this->controller_, std::move(cont
28 | rclexec::experimental::transform_each(
29 stdexec::then([this, mission, started](auto item) noexcept {
30 NavigateToPose::Feedback message;
31 message.navigation_time = this->clock_->now() - started;
32 message.distance_remaining = item->distance_to_goal;
33 mission->publish_feedback(std::make_shared<NavigateToPose::Feedback>(message)
34 }))
35 | stdexec::then(forward_unless_error<FollowPath::Result>)
36 | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; });
37
38 auto replan_task =
39 exec::schedule_after(this->sched_, 1s)
40 | stdexec::let_value([this, planner_goal] {
41 return rclexec::execute_action(this->planner, planner_goal);

```

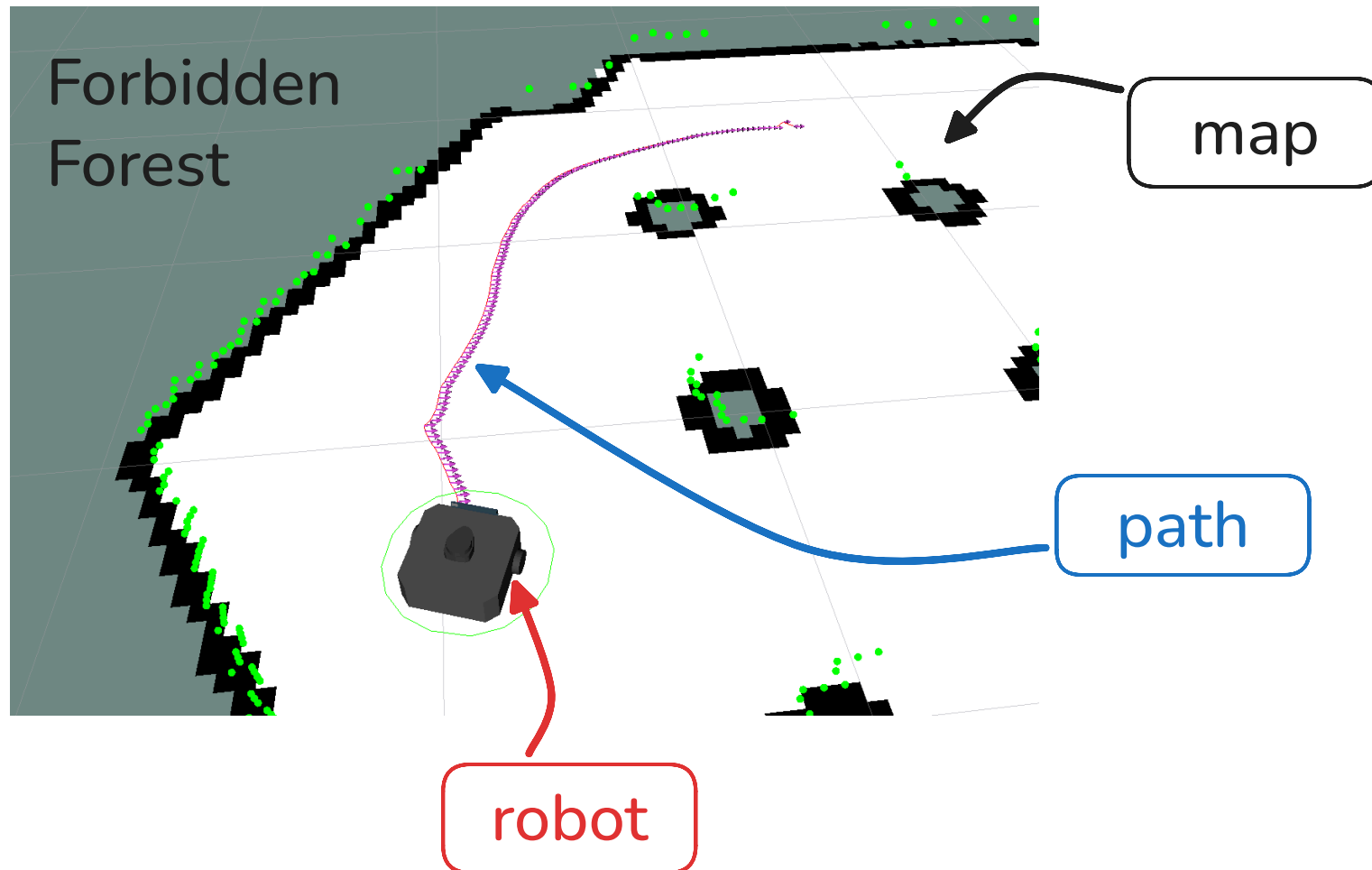
BONUS

SEQUENCE SENDERS

`rclexec::experimental::execute_action_with_feedback`

```
26     auto follow_task =
27         rclexec::experimental::execute_action_with_feedback(this->controller_, std::move(cont
28             | rclexec::experimental::transform_each(
29                 stdexec::then([this, mission, started](auto item) noexcept {
30                     NavigateToPose::Feedback message;
31                     message.navigation_time = this->clock_->now() - started;
32                     message.distance_remaining = item->distance_to_goal;
33                     mission->publish_feedback(std::make_shared<NavigateToPose::Feedback>(message)
34                         )))
35             | stdexec::then(forward_unless_error<FollowPath::Result>)
36             | stdexec::then([](auto&&) -> decltype(plan) { return nullptr; }));
37
38     auto replan_task =
39         exec::schedule_after(this->sched_, 1s)
40             | stdexec::let_value([this, planner_goal] {
41                 return rclexec::execute_action(this->planner_, planner_goal);
42             })
43             | stdexec::then(forward_unless_error<ComputePathToPose::Result>);
44
```

LIVE DEMO



*"Okay... RPC is fine. But I need to subscribe to a topic, do some expensive computation in a worker thread without blocking the event loop, and publish the result.
How does this help me?"*

- Someone (probably)

THANKS!

```
1 int main(int argc, char** argv) {
2   rclcpp::init(argc, argv, rclcpp::InitOptions(), rclcpp::SignalHandlerOptions::None);
3
4   auto node = std::make_shared<rclcpp::Node>("node");
5   auto node_sched = rclexec::timer_scheduler(node);
6
7   auto scope = stdexec::counting_scope{};
8
9   auto worker_loop = stdexec::run_loop{};
10  auto worker_sched = worker_loop.get_scheduler();
11
12  auto worker = std::jthread([&](std::stop_token token) {
13    // set up affinity / priority
14    auto cb = std::stop_callback(token, [&] { worker_loop.finish(); });
15    worker_loop.run();
16  });
17
18  auto pub = node->create_publisher<Result>("output_topic", 10);
19
20  auto sub = node->create_subscription<Data>("input_topic", 10, [&](auto msg) {
21    auto work = stdexec::just(std::move(msg))
22      | stdexec::continues_on(worker_sched)
23      | stdexec::then([](auto msg) { return expensive_computation(std::move(msg)); })
24      | stdexec::continues_on(node_sched)
25      | stdexec::then([pub](auto result) { pub->publish(result); })
26      | stdexec::upon_error([node](std::exception_ptr) {
27        RCLCPP_ERROR(node->get_logger(), "Something went wrong!");
28      });
29
30    stdexec::spawn(std::move(work), scope.get_token());
31  });
32
33  // Register a signal handler to request stop and sync wait for
34  // the scope to join before calling rclcpp::shutdown().
35  rclexec::spin_with_scope(node, scope);
36  return 0;
37 }
```

